Dafny Errors and Warnings

This page contains a catalog of the dafny tool's Error and Warning messages, all in one long page to facilitate searching.

The title of each section is an error message. Each section contains example code that produces the error, which is then followed by explanation and references.

Italicized words in the given messages indicate variable content.

Command-line Errors and Warnings

Error: No compiler found for target *target*; expecting one of *known*

```
dafny build -t:zzz
```

The `-t` or `--target` options specifies which backend compiler to use for those dafny commands that compile dafny to other programming languages. This error message says that the named language is not supported.

Error: no Dafny source files were specified as input

```
dafny resolve
```

Any valid dafny invocation requires at least one .dfy file. The only exceptions are the special commands like `--help` and `--version`.

*** Error: *file*: Central Directory corrupt.

```
dafny resolve --use-basename-for-filename testsource/test1.dfy testsource/BadDoo.
```

This error message says that the `.doo` file was not able to be successfully unzipped. It indicates that the file is either corrupted or misnamed (it should not be a .doo file).

Error: file *name* not found

```
dafny build -t:java testfiles/a.dfy zzz.java
```

This error message simply states that the named file could not be found by the dafny tool. Note that files on the command-line are named relative to the current working directory of the shell in which the command is invoked or are absolute file paths. In particular, this

instance of this error message identifies non-.dfy files that still have permitted extensions (according to the target platform) but cannot be found in the file system.

Error: command-line argument '–zzz' is neither a recognized option nor a Dafny input file (.dfy, .doo, or .toml).

```
dafny resolve --zzz
```

The named command-line argument is either a misspelled option or a filename in an unrecognized form (e.g., with no filename extension). An invalid option with a filename as value can look like either one, such as `--out:a.doo`.

Error: unknown switch: -zzz

```
dafny -zzz
```

The named command-line argument is either a misspelled option or a filename in an unrecognized form (e.g., with no filename extension).

Error: '*name*': Filename extension '*ext*' is not supported. Input files must be Dafny programs (.dfy) or supported auxiliary files (*ext*)

```
dafny build z.zzz
```

Dafny programs are in files with a `.dfy` extension. A Dafny program may be combined with other files appropriate to the target programming language being used, such as `.dll` files for C# or `.java` or `.jar` files for Java. The stated extension is not recognized.

Error: Only one .dfy file can be specified for testing

```
dafny generate-tests block x.dfy y.dfy
```

The test-generation tool is still experimental. However, it is expected to take just a single `.dfy` file as input.

Error: ModelView file must be specified when attempting counterexample extraction

```
dafny /extractCounterexample testsource/test1.dfy
```

When requesting a counterexample to be generated, some additional options must be specified: a model view file using `/mv:<file>` and `/proverOpt:O:model.completion=true` and `/proverOpt:O:model_compress=false` (for z3 version < 4.8.7) or `/proverOpt:O:model.compact=false` (for z3 version >= 4.8.7).

The model view file is an arbitrary non-existent file that is created and used as a temporary file during the creation of the counterexamples.

Error: Feature not supported for this compilation target: *feature*

```
method m() {}
```

The backends that Dafny supports have their own limitations on the ability to implement some of Dafny's features. This error message indicates that the Dafny program in question has hit one of these limitations. Some limitations are inherent in the language, but others are a question of implementation priorities to date. If the feature is an important one for your project, communicate with the Dafny team to determine if the omission can be corrected.

Please use the '–check' and/or '–print' option as stdin cannot be formatted in place.

```
dafny format --stdin
```

The dafny `format` command formats its input dafny program. The default behavior is to format the input files in place. It can instead just check that the file is formatted (`--check`) or print the formatted version to stdout (`--print`). If the `--stdin` option is used, the input dafny program is taken from stdin, in which case it cannot be formatted in place, so either the `--check` or `--print` options must be used in conjunction with `--stdin`.

The file *file* needs to be formatted

dafny format testsource/test2.dfy –>

This message is not an error. Rather it is the expected output of the `dafny format` command when `--check` is used: it states that the named input file is not formatted according to Dafny style. You can use `--print` to see the output. Also for an input file `test.dfy`, the command `dafny format --print test.dfy | diff test.dfy -` provides a diff between the original and the formatted files (without making any changes).

Error: Invalid argument *argument* to option print-mode_help_

```
dafny -printMode:zzz
```

The `printMode` option has a number of alternative values that are spelled out in the help document (`dafny -?`). It is also recommended to use the new CLI, with the option `--print-mode`.

Error: Unsupported diagnostic format: *format*; expecting one of 'json', 'text'.

```
dafny /diagnosticsFormat:zzz
```

This option controls the format of error messages. Typically the 'text' format is used (and is the default). But 'json' is also an option. Any other choice is illegal and results in this error message.

Error: Invalid argument *argument* to option functionSyntax_help_

```
dafny -functionSyntax:z
```

The `functionSyntax` option has a number of alternative values that are spelled out in the help document (`dafny -?`), most commonly the values '3' and '4' ('4' for dafny version 4.0 and later). It is also recommended to use the new CLI, with the option `--function-syntax`.

Error: Invalid argument *argument* to option quantifierSyntax_help_

```
dafny -quantifierSyntax:z
```

The `quantifierSyntax` option has a number of alternative values that are spelled out in the help document (`dafny -?`), most commonly the values '3' and '4' ('4' for dafny version 4.0 and later). It is also recommended to use the new CLI, with the option `--quantifier-syntax`.

Error: Invalid argument *argument* to option verificationLogger_help_

```
--log-format:z
```

The `--log-format` option has these alternatives: trx, csv, text.

Error: Invalid argument *argument* to option testContracts_help_

```
dafny -testContracts:z
```

The `testContracts` option has these alternatives: Externs, TestedExterns. The similar option in the new CLI is `--test-assumptions`.

Error: Invalid argument *argument* to option printIncludes_help_

```
dafny -printIncludes:z
```

The `printIncludes` option has these alternatives: None, Immediate, Transitive.

Argument '*argument*' not recognized. *Alternatives*

```
dafny resolve --show-snippets:false --quantifier-syntax:2 null.dfy
```

This is a generic error message about command-line arguments, indicating that the value given (after the `:` or `=` or as the next command-line argument) is not recognized as a valid value for the given option.

The correct spellings of the valid values should be stated in the help document (`dafny -?` or `dafny <command> -h`).

Error: unknown switch: *switch*

```
dafny --stdin
```

This error results from using an unknown command-line option in the legacy CLI, such as one that begins with a double hyphen. It is recommended to use the new CLI in which the command-line begins with an action verb (e.g., resolve, verify, run) followed by files, folders, and options written with double-hyphen prefixes.

Compilation Errors

Error: *process* Process exited with exit code *code*

The dafny compiler prepares a representation of the Dafny program in the desired target programming language and then invokes the target language's compiler on those files as a subsidiary process. This error message indicates that the sub-process terminated with an error of some sort. This should not ordinarily happen. Some possible causes are

- lack of resources that caused the underlying compiler to fail
- lack of or corrupted installation of the underlying compiler
- a bug in dafny whereby the source code it prepares for the compiler is faulty

There may be additional error messages from the underlying compiler that can help debug this situation. You can also attempt to run the target programming language compiler manually in your system to make sure it is functional.

Error: Unable to start *target*

The dafny compiler prepares a representation of the Dafny program in the desired target programming language and then invokes the target language's compiler on those files as a subsidiary process. This error message indicates that this sub-process would not start; it gives some additional information about what the invoking process was told about the failure to start.

Typically this error happens when the installation of the underlying compiler is not present or is incomplete, such as the proper files not having the correct permissions.

Error: Error: '*feature*' is not an element of the *target* compiler's UnsupportedFeatures set

This messages indicates two problems:

- the given feature is not supported in the compiler for the target language but is present in the program, so the program will need to be revised to avoid this feature;
- the feature is not listed in the in-tool list of unsupported features. The latter is an omission in the in-tool documentation. Please report this error message and the part of the program provoking it to the Dafny team's issue tracker.

Error: Abstract type (*name*) cannot be compiled; perhaps make it a type synonym or use :extern.

```
type T
```

Abstract types are declared like `type T`. They can be useful in programs where the particular characteristics of the type do not matter, such as in manipulating generic collections. Such programs can be verified, but in order to be compiled to something executable, the type must be given an actual definition. If the definition really does not matter, just give it a definition like `type T = bool` or `type T =` int`.

Note that some properties of a type an be indicated using a type characteristic. For example, `type T(00)` indicates that the type `T` is non-empty.

Error: since yield parameters are initialized arbitrarily, iterators are forbidden by the –enforce-determinism option

```
iterator Gen(start: int) yields (x: int)
  yield ensures |xs| <= 10 && x == start + |xs| - 1
{
  var i := 0;
  while i < 10 invariant |xs| == i {
    x := start + i;
    yield;
    i := i + 1;
  }
}
```

The `--enforce-determinism` option requires a target program to be deterministic and predictable: there may be no program statements that have an arbitrary, even if deterministic, result. In an iterator, the yield parameters are initialized with arbitary values and can be read before they are set with actual values, so there is a bit of nondeterminism. Consequently, iterators in general are not permitted along with `--enforce-determinism`.

Error: iterator *name* has no body

```
iterator Gen(start: int) yields (x: int)
  yield ensures |xs| <= 10 && x == start + |xs| - 1
```

Programs containing iterators without bodies can be verified. However, a body-less iterator is an unchecked assumption (even if it is ghost). Consequently, like body-less functions and loops, dafny will not compile a program containing an iterator without a body. Furthermore, if the iterator is non-ghost, it cannot be compiled if it does not have a body.

Error: since fields are initialized arbitrarily, constructor-less classes are forbidden by the –enforce-determinism option

```
class A { var j: int }
```

The `--enforce-determinism` option prohibits all non-deterministic operations in a Dafny program. One of these operations is the arbitrary values assigned to non-explicitly-initialized fields in a default constructor. Consequently an explicit constructor is required.

Error: The method '*name*' is not permitted as a main method (*reason*).

```
method mmm(i: int) {}
```

To create an executable program, the compiler needs to know which is the method to call to start the program, typically known as the 'main' method or main entry point. `dafny` chooses such a method based on a few rules (cf. details in the Dafny User Guide).

But there are restrictions on which methods may be used as a main entry point. Most importantly

- the method must take no input arguments or just one argument of type `seq<string>`
- it must not be ghost
- the method may be a class method, but then typically it is `static`

Most commonly and for clarity, the intended main method is marked with the attribute `{:main}`.

Error: Could not find the method named by the -Main option: *name*

```
class A { static method mm() {} }
```

In the legacy command-line syntax, the main method may be specified on the command-line. When doing so, the name given must be a fully-qualified name. For instance. a method `m` in a (top-level) module `M` is named `M.m` or in a class `C` in a module `M` is named `M.C.m`.

This error message indicates that dafny does not recognize the name given as the name of a method.

Error: More than one method is marked {:main}. First declaration appeared at *location*.

```
method {:main} M(s: seq<string>) {}
method {:main} P() {}
```

When searching all the files in the program (including `include`d ones), dafny found more than one method marked with `{:main}`. The error location is that of one of the methods and the error refers to another. The tool does not know which one to use. The solution is to remove the `{:main}` attribute from all but one.

Note that entry points that are intended as unit tests can be marked with `{:test}` instead.

Error: This method marked {:main} is not permitted as a main method (*name*).

```
method {:main} M(i: int) {}
```

Only some methods may be used as entry points and the one indicated may not. The principal restrictions are these (more details in the Dafny User Guide):

- the method must take no input arguments or just one argument of type `seq<string>`
- it must not be ghost
- the method may be a class method, but then typically it is `static`

Error: More than one method is declared as 'Main'. First declaration appeared at *location*.

```
method Main(s: seq<string>) {}
class A {
  static method Main() {}
}
```

In the absence of any `{:main}` attributes, dafny chooses a method named `Main` as the main entry point. When searching all the files in the program (including `include`d ones), dafny found more than one method named `Main`. The error location is that of one of the methods and the error text refers to another. The solution is to mark one of them as `{:main}`.

Note that entry points that are intended as unit tests can be marked with `{:test}` instead.

Error: This method 'Main' is not permitted as a main method (*reason*).

```
method Main(i: int) {}
```

If dafny finds no methods marked `{:main}`, it then looks for a method named `Main` to use as the starting point for the program. This error occurs if the `Main` method that is found does not qualify as a main entry point because it violates one or more of the rules, as given by the reason in the error message.

Error: Function *name* has no body so it cannot be compiled

```
function f(): int
```

The given function has no body, which is OK for verification, but not OK for compilation — the compiler does not know what content to give it. Even ghost functions must have bodies because body-less ghost functions are a form of unchecked assumptions.

Error: Function *name* must be compiled to use the {:test} attribute

```
ghost function {:test} f(): int { 42 }
```

Only compiled functions and methods may be tested using the `dafny test` command, which tests everything marked with `{:test}`. However this function is a ghost function, and consequently cannot be tested during execution. If you want the function to be compiled, declare the function as a `function` instead of `ghost function` in Dafny 4 or as `function method` instead of `function` in Dafny 3.

Error: Method *name* is annotated with :synthesize but is not static, has a body, or does not return anything

The `{:synthesize}` attribute is an experimental attribute used to create a mock object for methods that do not have bodies. It is currently only available for compiling to C# and in conjunction with the Moq library. See the reference manual section on {:synthesize} for more detail.

Error: Method *name* has no body so it cannot be compiled

```
method m()
```

To be part of a compiled program, each method must have a body. Ghost methods are the equivalent of unchecked assumptions so they too must have bodies.

Warning: assume statement has no {:axiom} annotation

```
method m(x: int) {
  assume x > 0;
```

```
}
```

Error: a forall statement without a body cannot be compiled

```
predicate P(i: int)
method m(a: array<int>) {
  forall x: int | P(x)
  var y := 3;
}
```

A method may be parsed and verified even if a forall statement is missing a body. However, the body must be supplied before the program can be compiled, even if the method is `ghost`. Body-less foralls in ghost methods are unchecked assumptions.

Error: a loop without a body cannot be compiled

```
ghost method m(i: int) {
  var x: int := i;
  while x > 0
  x := 3;
}
```

A method may be parsed and verified even if a loop is missing a body. Note that a missing body is different than an empty body, which is just `{ }`. However, the body must be supplied before the program can be compiled, even if the method is `ghost`. Body-less loops in ghost methods are similar to unchecked assumptions.

Error: nondeterministic assignment forbidden by the –enforce-determinism option

```
method m() {
  var x: int;
  x := *;
}
```

The `--enforce-determinism` option requires a target program to be deterministic and predictable: there may be no program statements that have an arbitrary, even if deterministic, result. Hence this 'assign any legal value' (`... := *`) statement is not permitted with `--enforce-determinism`.

There are a few different forms of this kind of assignment:

- `x := *;`
- `x, y, z := 3, *, 42;`
- `forall i | 0 <= i < a.Length { a[i] := *; }`

Error: assign-such-that statement forbidden by the –enforce-determinism option

```
method m() {
  var x: int;
  x :| x < 5;
}
```

The `--enforce-determinism` option requires a target program to be deterministic and predictable: there may be no program statements that have an arbitrary, even if deterministic, result. Hence this 'assign any value that satisfies the predicate' (`:|`) statement is not permitted with `--enforce-determinism`, even if there is only one such possible value. (The tool does not try to determine whether there is just one value and whether there is a reasonable way to compute the value.)

Error: this assign-such-that statement is too advanced for the current compiler; Dafny's heuristics cannot find any bound for variable '*name*'

To compile an assign-such-that statement, Dafny needs to find some appropriate bounds for each variable. However, in this case the expression is too complex for Dafny's heuristics.

Error: nondeterministic if statement forbidden by the –enforce-determinism option

```
method m() {
var y: int;
  if * { y:= 0; } else { y:= 1; }
}
```

The `--enforce-determinism` option requires a target program to be deterministic and predictable: there may be no program statements that have an arbitrary, even if deterministic, result. Hence this 'non-deterministic if' (`if *`) statement is not permitted with `--enforce-determinism`.

Error: binding if statement forbidden by the –enforce-determinism option

```
method m(k: int) {
  var i := k;
  var y: int;
  if i :| i < 5 {  } else { y := 1; }
}
```

The `--enforce-determinism` option requires a target program to be deterministic and predictable: there may be no program statements that have an arbitrary, even if

deterministic, result. The `if` with a binding guard (`if ... :|`) checks that the given predicate is satisfiable by some value. If not, then the 'else' branch is executed; but if so, the 'then' branch is executed with an arbitrary value that satisifies the predicate. Because of this arbitrary selection, the if-with-binding-guard is not permitted with `--enforce-determinism`, even if there is exactly one value that satisfies the predicate. (The tool does not try to determine whether there is just one value or whether there is a reasonable way to compute a value.)

Error: case-based if statement forbidden by the –enforce-determinism option

```
method m(k: int) {
  var i := k;
  if
  {
    case i >= 0 => i := i - 1;
    case i <= 0 => i := i + 1;
  }
}
```

The `--enforce-determinism` option requires a target program to be deterministic and predictable: there may be no program statements that have an arbitrary, even if deterministic, result. The case-based if statement allows the case guards to be evaluated in an arbitrary order and an arbitrary one of those found to be true is chosen to be executed. Hence the case-based if is not permitted with `--enforce-determinism`.

To enforce a deterministic order to the evaluation, use a chain of if-then-else statements.

Error: nondeterministic loop forbidden by the –enforce-determinism option

```
method m(b: bool) decreases * {
  while *
    decreases *
  { }
}
```

The `--enforce-determinism` option requires a target program to be deterministic and predictable: there may be no program statements that have an arbitrary, even if deterministic, result. Hence this 'non-deterministic while' (`while *`) statement is not permitted with `--enforce-determinism`.

Error: case-based loop forbidden by the –enforce-determinism option

```
method m(k: int) {
  var i := k;
  while
    decreases if i < 0 then -i else i
  {
    case i > 0 => i := i - 1;
    case i < 0 => i := i + 1;
  }
}
```

The `--enforce-determinism` option requires a target program to be deterministic and predictable: there may be no program statements that have an arbitrary, even if deterministic, result. The case-based while statement allows the case guards to be evaluated in an arbitrary order and an arbitrary one of those found to be true is chosen to be executed. Hence the case-based loop is not permitted with `--enforce-determinism`.

To enforce a deterministic order to the evaluation, use a chain of if-then-else statements or series of `if` statements in which the then-branch ends in a continue statement.

Error: compiler currently does not support assignments to more-than-6-dimensional arrays in forall statements

The documentation of this problem is in progress.

Error: modify statement without a body forbidden by the –enforce-determinism option

```
class A { constructor A(){}}

method m(k: int, o: A)
  modifies o
{
  var i := k;
  modify o;
}
```

The `--enforce-determinism` option requires a target program to be deterministic and predictable: there may be no program statements that have an arbitrary, even if deterministic, result.

The `modify` statement allows the program to modify the state of the given objects in an arbitrary way (though each field still has a legal value of its declared type). Hence such a statement is not permitted with `--enforce-determinism`.

Note that a `modify` statement with a body is deprecated.

Error: this let-such-that expression is too advanced for the current compiler; Dafny's heuristics cannot find any bound for variable '*name*'

The documentation of this problem is in progress.

**Error: Post-conditions on function *function* might be unsatisfied when synthesizing code for method *name*"

This message relates to mocking methods in C# with the Moq framework. See the reference manual section on {:synthesize} for more detail.

Error: Stubbing fields is not recommended (field *name* of object *object* inside method *method*)

This message relates to mocking methods in C# with the Moq framework. See the reference manual section on {:synthesize} for more detail.

Error: Abstract type ('*type*') with unrecognized extern attribute *attr* cannot be compiled. Expected {:extern compile_type_hint}, e.g., 'struct'.

```
type {:extern "class" } T
```

The C++ compiler must be told how to translate an abstract type. Typically the value of the extern attribute is "struct". Note that Dafny's C++ compiler is very preliminary, partial and experimental.

Errors specific to the Go compiler

Error: Unsupported field *name* in extern trait

Documentation of the Go compiler errors is in progress.

Error: Cannot convert from *type* to *target-type*

Documentation of the Go compiler errors is in progress.

Parser Errors and Warnings

Error: Duplicate declaration modifier: abstract

```
abstract abstract module M {}
```

No Dafny modifier, such as `abstract`, `static`, `ghost` may be repeated Such repetition would be superfluous even if allowed.

Error: a *decl* cannot be declared 'abstract'

```
abstract const c := 4
```

Only modules may be declared abstract.

Error: a function-by-method has a ghost function body and a non-ghost method body; a function-by-method declaration does not use the 'ghost' keyword.

```
ghost function f(): int
{
  42
} by method {
  return 42;
}
```

Functions with a by method section to their body can be used both in ghost contexts and in non-ghost contexts; in ghost contexts the function body is used and in compiled contexts the by-method body is used. The `ghost` keyword is not permitted on the declaration.

Error: *decl* cannot be declared 'ghost' (it is 'ghost' by default when using –function-syntax:3)

```
module {:options "--function-syntax:3"} M {
  ghost function f(): int { 42 }
}
```

For versions prior to Dafny 4, the `function` keyword meant a ghost function and `function method` meant a non-ghost function. From Dafny 4 on, `ghost function` means a ghost function and `function` means a non-ghost function. See the discussion here for a discussion of options to control this feature.

Error: *decl* cannot be declared 'ghost' (it is 'ghost' by default)

```
module {:options "--function-syntax:4"} M {
  ghost least predicate p()
}
```

For versions prior to Dafny 4, the `function` keyword meant a ghost function and `function method` meant a non-ghost function. From Dafny 4 on, `ghost function` means a ghost function and `function` means a non-ghost function. See the discussion here for a discussion of options to control this feature.

Error: a *decl* cannot be declared 'ghost'

```
ghost module M {}
```

Only some kinds of declarations can be declared `ghost`, most often functions, fields, and local declarations.

Error: a *decl* cannot be declared 'static'

```
static module M {}
```

Only some kinds of declarations can be declared 'static', most often fields, constants, methods, and functions, and only within classes. Modules and the declarations within them are already always static.

Error: a *decl* cannot be declared 'opaque'

```
opaque module M {}
```

Only some kinds of declarations can be declared 'opaque': const fields and the various kinds of functions.

**Warning: attribute *attribute* is deprecated

This attribute is obsolete and unmaintained. It will be removed from dafny in the future.

Error: argument to :options attribute must be a literal string

```
module N { const opt := "--function-syntax:4" }
import opened N
module {:options opt} M{}
```

The value of an options attribute cannot be a computed expression. It must be a literal string.

Error: cannot declare identifier beginning with underscore

```
function m(): (_: int) {0}
```

User-declared identifiers may not begin with an underscore; such identifiers are reserved for internal use. In match statements and expressions, an identifier that is a single underscore is used as a wild-card match.

Error: sorry, bitvectors that wide (*number*) are not supported

```
const b: bv2147483648
```

A bitvector type name is the characters 'bv' followed by a non-negative integer literal. However, dafny only supports widths up to the largest signed 32-bit integer (2147483647).

Error: sorry, arrays of that many dimensions (*number*) are not supported

```
const a: array2147483648<int>
```

A multi-dimensional array type name is the characters 'array' followed by a positive integer literal. However, dafny only supports numbers of dimensions up to the largest signed 32-bit integer (2147483647). In practice (as of v3.9.1) dimensions of more than a few can take overly long to resolve (Issue #3180).

Error: There is no point to an export declaration at the top level

```
export M
method M() {}
```

Although all top-level declarations are contained in an implicit top-level module, there is no syntax to import that module. Such an import would likely cause a circular module dependency error. If the implicit module cannot be imported, there is no point to any export declarations inside the implicit module.

Error: expected either a '{' or a 'refines' keyword here, found *token*

```
module M {}
module N refine M {}
```

The syntax for a module declaration is either `module M { ... }` or `module M refines N { ... }` with optional attributes after the `module` keyword. This error message often occurs if the `refines` keyword is misspelled.

Warning: the *name* token is the identifier for the export set, not an adjective for an extreme predicate

```
module M {
  export
  least predicate p()
}
```

A `least` or `greatest` token between `export` and `predicate` is a bit ambiguous: it can be either the name of the export set or associated with the `predicate` declaration. The parser associates it with the `export`. To avoid this warning, do not put the `least` or `greatest` token on the same line as the `predicate` token. If you intend for the `least` to go with the predicate, change the order of the declarations.

Error: no comma is allowed between provides and reveals and extends clauses in an export declaration

```
module M {
  export A reveals b, a, reveals b
  export B reveals b, a, provides b
  export C provides b, a, reveals b
  export D provides b, a, provides b
  const a: int
  const b: int
}
```

An export declaration consists of one or more `reveals`, `provides`, and extends clauses. Each clause contains a comma-separated list of identifiers, but the clauses themselves are not separated by any delimiter. So in the example above, the comma after `a` is wrong in each export declaration. This mistake is easy to make when the clauses are on the same line.

Error: fields are not allowed to be declared at the module level; instead, wrap the field in a 'class' declaration

```
module M {
  var c: int
}
```

`var` declarations are used to declare mutable fields of classes, local variables in method bodies, and identifiers in let-expressions. But mutable field declarations are not permitted at the static module level, including in the (implicit) toplevel module. Rather, you may want the declaration to be a `const` declaration or you may want the mutable field to be declared in the body of a class.

Warning: module-level functions are always non-instance, so the 'static' keyword is not allowed here

```
static predicate p() { true }
```

All names declared in a module (outside a class-like entity) are implicitly `static`. Dafny does not allow them to be explictly, redundantly, declared `static`.

Warning: module-level methods are always non-instance, so the 'static' keyword is not allowed here

```
static method m() {}
```

All names declared in a module (outside a class-like entity) are implicitly `static`. Dafny does not allow them to be explictly, redundantly, declared `static`.

Error: in refining a datatype, the '...' replaces the '=' token and everything up to a left brace starting the declaration of the body; only members of the body may be changed in a datatype refinement

```
abstract module M { datatype D = A | B }
module N refines M { datatype D = ... Y | Z }
```

There are limitations on refining a datatype, namely that the set of constructors cannot be changed. It is only allowed to add members to the body of the datatype.

Error: datatype extending traits is not yet enabled by default; use –general-traits=datatype to enable it

```
trait Trait { }
datatype D extends Trait = A | B
```

A newtype extending a trait is not generally supported. The option –general-traits=full causes Dafny to allow them in the input, but is not recommended.

Error: newtype extending traits is not fully supported (specifically, compilation of such types is not supported); to use them for verification only, use –general-traits=full

```
trait Trait { }
newtype N extends Trait = int
```

Use of traits as non-reference types is supported, but is not yet the default. Until it becomes the default, use –general–traits=datatype to enable it.

**Warning: module-level const declarations are always non-instance, so the 'static' keyword is not allowed here

```
static const i := 9
```

All names declared in a module (outside a class-like entity) are implicitly `static`. Dafny does not allow them to be explictly, redundantly, declared `static`.

Error: expected an identifier after 'const' and any attributes

```
const c := 5
const
```

```
const d := 5
```

This error arises from a truncated declarations of a const field, namely just a const keyword. To correct the error, add an identifier and either or both a type and initializing expression (or remove the const keyword).

Error: a const field should be initialized using ':=', not '='

```
const c: int = 5
```

Dafny's syntax for initialization of const fields uses `:=`, not `=`. In Dafny, `=` is used only in type definitions.

Error: a const declaration must have a type or a RHS value

```
const i
```

A `const` declaration needs its type indicated by either an explicit type or a right-hand-side expression, whose type is then the type of the declared identifier. So use syntax either like `const i: int` or `const i := 5` (or both together).

Error: in refining a newtype, the '...' replaces the '=' token and everything up to the left brace starting the declaration of the newtype body (if any); a newtype refinement may not change the base type of the newtype

```
abstract module M { newtype T = int }
module N refines M { newtype T = ... int }
```

There are limitations on refining a newtype, namely that the base type cannot be changed. You can change an abstract type into a newtype, however. When refining a newtype by adding a body, the ... stands in place of both the '=' and the base type.

Error: formal cannot be declared 'ghost' in this context

```
twostate function p(i: int): (ghost r: int) { true }
```

The output of a predicate or function cannot be ghost. It is implicitly ghost if the function is ghost itself.

Error: formal cannot be declared 'new' in this context

```
method m(i: int) returns (new r: int) { r := 0; }
```

The `new` modifier only applies to input parameters.

Error: formal cannot be declared 'nameonly' in this context

```
method m(i: int) returns (nameonly r: int) { r := 0; }
```

The `nameonly` modifier only applies to input parameters.

Error: formal cannot be declared 'older' in this context

```
method m(i: int) returns (older r: int) { r := 0; }
```

The `older` modifier only applies to input parameters.

Error: a mutable field must be declared with a type

```
class A {
  var f
  const g := 5
}
```

Because a mutable field does not have initializer, it must have a type (as in `var f: int`). `const` declarations may have initializers; if they do they do not need an explicit type.

Error: a mutable field may not have an initializer

```
class A {
  var x: int := 6
  var y: int, x: int := 6, z: int
}
```

Dafny does not allow field declarations to have initializers. They are initialized in constructors. Local variable declarations (which also begin with `var`) may have initializers.

Error: invalid formal-parameter name in datatype constructor

```
datatype D = Nil | D(int: uint8)
```

Datatype constructors can have formal parameters, declared with the usual syntax: 'name: type'. In datatype constructors the 'name :' is optional; one can just state the type. However, if there is a name, it may not be a typename. The formal parameter name should be a simple identifier that is not a reserved word.

Error: use of the 'nameonly' modifier must be accompanied with a parameter name

```
datatype D = D (i: int, nameonly int) {}
```

The parameters of a datatype constructor do not need to have names – it is allowed to just give the type. However, if `nameonly` is used, meaning the constructor can be called using named parameters, then the name must be given, as in `datatype D = D (i: int, nameonly j: int) {}`

More detail is given here.

Error: iterators don't have a 'returns' clause; did you mean 'yields'?

```
iterator Count(n: nat) returns (x: nat) {
  var i := 0;
  while i < n {
    x := i;
    yield;
    i := i + 1;
  }
}
```

An iterator is like a co-routine: its control flow produces (yields) a value, but the execution continues from that point (a yield statement) to go on to produce the next value, rather than exiting the method. To accentuate this difference, a `yield` statement is used to say when the next value of the iterator is ready, rather than a `return` statement. In addition, the declaration does not use `returns` to state the out-parameter, as a method would. Rather it has a `yields` clause. The example above is a valid example if `returns` is replaced by `yields`.

Error: type-parameter variance is not allowed to be specified in this context

```
type List<T>
method m<+T>(i: int, list: List<T>) {}
method m<T,-U>(i: int, list: List<T>) {}
```

Type-parameter variance is specified by a `+`, `-`, `*` or `!` before the type-parameter name. Such designations are allowed in generic type declarations but not in generic method, function, or predicate declarations.

Error: unexpected type characteristic: '000' should be one of == or 0 or 00 or !new

```
type T(000)
```

Type characteristics, indicated in parentheses after the type name, state properties of the otherwise uninterpreted or abstract type. The currently defined type characteristics are designated by `==` (equality-supporting), `0` (auto-initializable), `00` (non-empty), and `!new` (non-reference).

Error: extra comma or missing type characteristic: should be one of == or 0 or 00 or !new

```
type T(0,,0)
```

Type characteristics, state properties of the otherwise uninterpreted or abstract type. They are given in a parentheses-enclosed, comma-separated list after the type name. The currently defined type characteristics are designated by `==` (equality - supporting), `0` (auto - initializable), `00` (non - empty), and `!new` (non - reference).

Error: illegal type characteristic: '*token*' should be one of == or 0 or 00 or !new

```
type T(X)
```

Type characteristics, indicated in parentheses after the type name, state properties of the otherwise uninterpreted or abstract type. The currently defined type characteristics are designated by `==` (equality - supporting), `0` (auto - initializable), `00` (non - empty), and `!new` (non - reference). Type characteristics are given in a parentheses-enclosed, comma-separated list after the type name.

Warning: the old keyword 'colemma' has been renamed to the keyword phrase 'greatest lemma'

```
colemma m() ensures true {}
```

The adjectives `least` and `greatest` for lemmas and functions are more consistent with the nomenclature for coinduction.

Warning: the old keyword phrase 'inductive lemma' has been renamed to 'least lemma'

```
inductive lemma m() ensures true {}
```

The adjectives `least` and `greatest` for lemmas and functions are more consistent with the nomenclature for coinduction.

Error: constructors are allowed only in classes

```
module M {
  constructor M() {}
}
```

Constructors are methods that initialize class instances. That is, when a new instance of a class is being created, using the `new` object syntax, some constructor of the class is called, perhaps a default anonymous one. So, constructor declarations only make sense within classes.

Error: a method must be given a name (expecting identifier)

```
method {:extern "M"} (i: int) {}
```

A method declaration always requires an identifier between the `method` keyword and the `(` that starts the formal parameter list. This is the case even when, as in the example above, a name is specified using `:extern`. The extern name is only used in the compiled code; it is not the name used to refer to the method in Dafny code

Error: type of *k* can only be specified for least and greatest lemmas

```
lemma b[nat](i: int) { }
```

Least and greatest lemmas and predicates have a special parameter named `k`. Its type is specified in square brackets between the lemma/predicate name and the rest of the signature. The type may be either `nat` or `ORDINAL`. But this type is specified only for `least` and `greatest` constructs.

Error: constructors cannot have out-parameters

```
class C {
  constructor (i: int) returns (j: int) {}
}
```

Constructors are used to initalize the state of an instance of a class. Thus they typically set the values of the fields of the class instance. Constructors are used in `new` object expressions, which return a reference to the newly constructed object (as in `new C(42)`). There is no syntax to receive out-parameter values of a constructor and they may not be declared. (This is similar to constructors in other programming languages, like Java.)

Error: A 'reads' clause that contains '*' is not allowed to contain any other expressions

```
const a: object
function f(): int
```

```
  reads a, *
{
  0
}
```

A reads clause lists the objects whose fields the function is allowed to read (or expressions containing such objects). `reads *` means the function may read anything. So it does not make sense to list `*` along with something more specific. If you mean that the function should be able to read anything, just list `*`. Otherwise, omit the `*` and list expressions containing all the objects that are read.

Error: out-parameters cannot have default-value expressions

```
method m(i: int) returns (j: int := 0) { return 42; }
```

Out-parameters of a method are declared (inside the parentheses after the `returns` keyword) with just an identifier and a type, separated by a colon. No initializing value may be given. If a default value is needed, assign the out-parameter that value as a first statement in the body of the method.

Error: set type expects only one type argument

```
const c: set<int,bool>
```

A `set` type has one type parameter, namely the type of the elements of the set. The error message states that the parser sees some number of type parameters different than one. The type parameters are listed in a comma-separated list between `<` and `>`, after the type name.

Error: iset type expects only one type argument

```
const c: iset<int,bool>
```

A `iset` type has one type parameter, namely the type of the elements of the set. The error message states that the parser sees some number of type parameters different than one. The type parameters are listed in a comma-separated list between `<` and `>`, after the type name.

Error: multiset type expects only one type argument

```
const c: multiset<int,bool>
```

A `multiset` type has one type parameter, namely the type of the elements of the multiset. The error message states that the parser sees some number of type parameters different

than one. The type parameters are listed in a comma-separated list between `<` and `>`, after the type name.

Error: seq type expects only one type argument

```
const c: seq<int,bool>
```

A `seq` type has one type parameter, namely the type of the elements of the sequence. The error message states that the parser sees some number of type parameters different than one. The type parameters are listed in a comma-separated list between `<` and `>`, after the type name.

Error: map type expects two type arguments

```
const m: map<int,bool,string>
```

A `map` type has two type parameters: the type of the keys and the type of the values. The error message states that the parser sees some number of type parameters different than two.

Error: imap type expects two type arguments

```
const m: imap<int,bool,string>
```

A `imap` type has two type parameters: the type of the keys and the type of the values. The error message states that the parser sees some number of type parameters different than two.

Error: arrow-type arguments may not be declared as 'ghost'

```
const f: (ghost int, bool) -> int
```

Arrow types are the types of functions in Dafny. They are designated with the arrow syntax, as shown in the example, except that the types used cannot be declared as ghost.

Error: empty type parameter lists are not permitted

An instantiation of a generic type consists of the generic type name followed by a comma-separated list of type arguments enclosed in angle brackets. If a type has no type arguments, then there is no list and no angle brackets either.

However, this particular error message is not reachable in the current parser. If the message is seen, please report the code that caused it so that the bug or documentation can be corrected.

Error: a formal [ ] declaration is only allowed for least and greatest predicates

```
predicate p[nat]() { true }
```

A declaration of an extreme predicate includes a special formal in addition to the regular parenthesis-enclosed list of formals. The type of that special formal is given in square brackets between the predicate name and the opening parenthesis of the formals. The type may be either `nat` or `ORDINAL`. This special formal is not permitted in a regular (non-extreme) predicate.

**Warning: the old keyword phrase 'inductive predicate' has been renamed to 'least predicate'

```
inductive predicate p()
```

The terms `least predicate` and `greatest predicate` are more descriptive of the relationship between them than was the old terminology.

**Warning: the old keyword 'copredicate' has been renamed to the keyword phrase 'greatest predicate'

```
copredicate p()
```

The terms `least predicate` and `greatest predicate` are more descriptive of the relationship between them than was the old terminology.

Error: a 'by method' implementation is not allowed on a twostate *what*

```
class Cell { var data: int  constructor(i: int) { data := i; } }
twostate predicate Increasing(c: Cell)
  reads c
{
  old(c.data) <= c.data
} by method {
  return old(c.data) <= c.data;
}
```

Two state functions and predicates are always ghost and do not have a compiled representation. Such functions use values from two different program (heap) states, which is not something that can be implemented (at least with any degree of good performance) in conventional programming languages.

Because there is no feasible compiled verion of a two-state function, it cannot have a `by method` block (which is always compiled).

Error: a 'by method' implementation is not allowed on an extreme predicate

```
least predicate g() { 42 } by method { return 42; }
```

Least and greatest predicates are always ghost and do not have a compiled representation, so it makes no sense to have a `by method` alternative implementation.

Error: to use a 'by method' implementation with a *what*, declare *id* using *what*, not '*what* method'

```
function method m(): int {
  42
} by method {
  return 42;
}
```

`by method` constructs combine a ghost function (or predicate) with a non-ghost method. The two parts compute the same result, and are verified to do so. Uses of the function are verified using the function body, but the method body is used when the function is compiled.

Thus the function part is always implicitly `ghost` and may not be explicitly declared `ghost`.

Error: a *adjective what* is supported only as ghost, not as a compiled *what*

```
twostate function method m(): int {
  42
}
```

The `twostate`, `least`, and `greatest` functions and predicates are always ghost and cannot be compiled, so they cannot be declared as a `function method` (v3 only).

Error: a *what* is always ghost and is declared with '*what*'

```
module {:options "--function-syntax:experimentalPredicateAlwaysGhost"} M {
  predicate method p() { true }
}
```

This error only occurs when the `experimentalPredicateAlwaysGhost` option is chosen. It indicates that `predicates` are always ghost and cannot be declared with the (Dafny 3) syntax `predicate method`.

- If you intend to predicate to be ghost, remove `method`.
- If you intend the predicate to be non-ghost, you either cannot use `experimentalPredicateAlwaysGhost` or you should use `function` with a `bool` return type instead of `predicate`

Error: the phrase '*what* method' is not allowed when using –function-syntax:4; to declare a compiled predicate, use just 'predicate'

```
module {:options "--function-syntax:4"} M {
  predicate method f() { true }
}
```

From Dafny 4 on, the phrases `function method` and `predicate method` are no longer accepted. Use `function` for compiled, non-ghost functions and `ghost function` for non-compiled, ghost functions, and similarly for predicates. See the documentation here.

Error: the phrase '*what* method' is not allowed when using –function-syntax:4; to declare a compiled function, use just 'function'

```
module {:options "--function-syntax:4"} M {
  function method f(): int { 42 }
}
```

From Dafny 4 on, the phrases `function method` and `predicate method` are no longer accepted. Use `function` for compiled, non-ghost functions and `ghost function` for non-compiled, ghost functions, and similarly for predicates. See the documentation here.

Error: there is no such thing as a 'ghost predicate method'

```
module {:options "--function-syntax:experimentalDefaultGhost"} M {
  ghost predicate method f() { true }
}
```

Pre-Dafny 4, a `function method` and a `predicate method` are explicitly non-ghost, compiled functions, and therefore cannot be declared `ghost` as well. If indeed the function or predicate is intended to be ghost, leave out `method`; if it is intended to be non-ghost, leave out `ghost`.

From Dafny 4 on, a ghost function is declared `ghost function` and a non-ghost function is declared `function` and there is no longer any declaration of the form `function method`, and similarly for predicates.

See the documentation here.

Error: there is no such thing as a 'ghost function method'

```
module {:options "--function-syntax:experimentalDefaultGhost"} M {
  ghost function method f(): int { 42 }
}
```

Pre-Dafny 4, a `function method` and a `predicate method` are explicitly non-ghost, compiled functions, and therefore cannot be declared `ghost` as well. If indeed the function or predicate is intended to be ghost, leave out `method`; if it is intended to be non-ghost, leave out `ghost`.

From Dafny 4 on, a ghost function is declared `ghost function` and a non-ghost function is declared `function` and there is no longer any declaration of the form `function method`, and similarly for predicates.

See the documentation here.

Error: a *what* must be declared as either 'ghost *what*' or '*what* method' when using –function-syntax:migration3to4

```
module {:options "--function-syntax:migration3to4"} M {
  function f(): int { 42 }
}
```

This error occurs only when using `migration3to4`. With this option, ghost functions are declared using `ghost function` and compiled functions using `function method`. Change `function` in the declaration to one of these.

Error: formal cannot be declared 'ghost' in this context

```
ghost predicate p(ghost i: int) { true }
```

A ghost predicate or function effectively has all ghost formal parameters, so they cannot be declared ghost in addition.

Error: 'decreases' clauses are meaningless for least and greatest predicates, so they are not allowed

```
least predicate m(i: int)
  decreases i
{
  true
}
```

Least and greatest predicates are not checked for termination. In fact, unbounded recursion is part of being coinductive. Hence `decreases` clauses are inappropriate and not allowed.

Error: *name* return type should be bool, got *type*

```
predicate b(): (r: boolean) { 4 }
```

A predicate is a function that returns `bool`. The return type here is something else. If you mean to have a non-`bool` return type, use `function` instead of `predicate`.

Error: _name_s do not have an explicitly declared return type; it is always bool. Unless you want to name the result: ': (result: bool)'

```
predicate p(): bool { true }
```

A `predicate` is simply a `function` that returns a `bool` value. Accordingly, the type is (required to be) omitted, unless the result is being named. So `predicate p(): (res: bool) { true }` is permitted.

Error: A '*' expression is not allowed here

```
function m(i: int): int
  decreases *
{
  42
}
```

A method or loop with a `decreases *` clause is not checked for termination. This is only permitted for non-ghost methods and loops. Insert an actual decreases expression.

Error: A '*' frame expression is not permitted here

```
iterator Gen(start: int) yields (x: int)
  reads *
{
  var i := 0;
  while i < 10 invariant |xs| == i {
    x := start + i;
    yield;
    i := i + 1;
  }
}
```

A `reads *` clause means the reads clause allows the functions it specifies to read anything. Such a clause is not allowed in an iterator specification. Insert a specific reads expression.

Warning: *kind* refinement is deprecated

Statement refinement has been deprecated. Refinement is restricted to changing declarations, not bodies of methods or functions.

Error: invalid statement beginning here (is a 'label' keyword missing? or a 'const' or 'var' keyword?)

```
method m(n: nat) {
  x: while (0 < n) { break x; }
}
```

In this situation the parser sees the identifier (x) and the following colon. This is not a legal start to a statement. Most commonly either

- a `var` or `const` keyword is missing, and the `x:` is the beginning of a declaration, or
- a `label` keyword is missing and the identifier is the label for the statement that follows. (The second error is somewhat common because in C/C++ and Java, there is no keyword introducing a label, just the identifier and the colon.)

Error: An initializing element display is allowed only for 1-dimensional arrays

```
method m() {
  var a := new int[2,2] [1,2,3,4];
}
```

One-dimensional arrays can be initialized like `var s := new int[][1,2,3,4];`, but multi-dimensional arrays cannot. The alternatives are to initialize the array in a loop after it is allocated, or to initialize with a function, as in `var a:= new int[2,2]((i: int, j: int)=>i+j)`.

Error: a local variable should be initialized using ':=', ':-', or ':|', not '='

```
method m() {
  var x = 5;
}
```

Local variables are initialized with `:=` (and sometimes with `:-` or `:|`), but not with `=`. In Dafny `=` is used only in type definitions.

Error: LHS of assign-such-that expression must be variables, not general patterns

```
datatype D = D(x: int, y: int)
method m() {
```

```
  var D(x,y) :| x + y == 5;
}
```

The `:|` syntax is called *assign-such-that*: the variables on the left-hand-side are initialized or assigned some non-deterministic values that satisfy the predicate on the right-hand-side.

However, Dafny only allows a list of simple variables on the left, not datatype deconstructor patterns, as seen here.

Error: 'modifies' clauses are not allowed on refining loops

Refining statements, including loops, are deprecated.

Error: Expected 'to' or 'downto'

```
method m(n: nat) {
  for i := n DownTo 0 {}
}
```

A for loop can express a computation to be performed for each value of a *loop index*. In Dafny, the loop index is an int-based variable that is either

- incremented up from a starting value to just before an ending value: `3 to 7` means 3, 4, 5, and 6
- or decremented from just below a starting value down to an ending value: `7 downto 3` means 6, 5, 4, and 3.

The contextual keywords `to` and `downto` indicate incrementing and decrementing, respectively. No other words are allowed here, including writing them with different case.

These two words have special meaning only in this part of a for-loop; they are not reserved words elsewhere. That is, the code

```
method m() {
  var to: int := 6;
  var downto: int := 8;
  for to := to to downto {}
}
```

is legal, but not at all recommended.

Error: A 'decreases' clause that contains '*' is not allowed to contain any other expressions

```
method f(n: int) returns (r: int)
  decreases *, n
{
```

```
  r := if n == 0 then n else -1-f(n-1);
}
```

A `decreases` clause provides a metric that must decrease from one call to the next, in order to assure termination of a sequence of recursive calls. In loops it assures termination of the loop iteration.

`decreases *` means to skip the termination check. So it does not make sense to both list a metric and to list '*'. Use one or the other.

Error: expected either 'by' or a semicolon following the assert expression

```
method m() {
  assert true
}
```

Assert statements, like all statements, end in either a semicolon or a block. Most assert statements end in semicolons, but an assert-by statement has the form `assert expr by { ... }` where the by-block contains statements such as lemma calls that assist in proving the validity of the asserted expression.

Warning: a forall statement with no bound variables is deprecated; use an 'assert by' statement instead

Warning: the modify statement with a block statement is deprecated

Error: the main operator of a calculation must be transitive

```
lemma abs(a: int, b: int, c: int)
  ensures a + b + c == c + b + a
{
  calc != {
    a + b + c;
    a + c + b;
    c + a + b;
  }
}
```

A calc statement contains a sequence of expressions interleaved by operators. Such a sequence aids the verifier in establishing a desired conclusion. But the sequence of operators must obey certain patterns similar to chaining expressions. In this case a default operator is stated (the `!=` between `calc` and `{`). This default operator is the implicit operator between each consecutive pair of expressions in the body of the calc statement.

But the operator has to be transitive: `!=` is not allowed; `==`, `<`, `<=`, '>' and '>=' are allowed.

Error: this operator cannot continue this calculation

```
lemma abs(a: int, b: int, c: int)
  ensures a + b + c == c + b + a
{
  calc {
    a + b + c;
    !=
    a + c + b;
    !=
    c + a + b;
  }
}
```

A calc statement contains a sequence of expressions interleaved by operators. Such a sequence aids the verifier in establishing a desired conclusion. But the sequence of operators must obey certain patterns similar to chaining expressions.

In particular, this error message is complaining that it sees an unacceptable operator. In this case, the reason is that the sequence may contain only one `!=` operator; another reason causing this message would be a combination of `<` and `>` operators.

Error: a calculation cannot end with an operator

```
lemma abs(a: int, b: int, c: int)
  ensures a + b + c == c + b + a
{
  calc {
    a + b + c;
    ==
  }
}
```

A calc statement contains a sequence of expressions interleaved by operators. Such a sequence aids the verifier in establishing a desired conclusion. But the sequence must begin and end with (semicolon-terminated) expressions.

This error message is complaining that it sees an operator ending the sequence. This may be because there is no following expression or that the parser does not recognize the material after the last operator as a legal ending expression.

Error: Calls with side-effects such as constructors are not allowed in expressions.

```
class A { function f(): int { 0 } }
const c := (new A).f()
```

Dafny expressions may not have side-effects. This prohibits both assignments to local variables and any changes to the heap. Thus method and constructor calls may not occur in expressions. This check is syntactic, so even methods that do not modify anything are not permitted in expressions.

Error: Ambiguous use of ==> and <==. Use parentheses to disambiguate.

```
const b := true <== false ==> true
```

The `==>` and `<==` operators have the same precedence but do not associate with each other. You must use parentheses to show how they are grouped. Write `p ==> q <== r` as either `(p ==> q) <== r` or `p ==> (q <== r)`.

In contrast, `p ==> q ==> r` is `p ==> (q ==> r)` and `p <== q <== r` is `(p <== q) <== r`.

See this section for more information.

Error: Ambiguous use of ==> and <==. Use parentheses to disambiguate.

```
const b := true ==> false <== true
```

The `==>` and `<==` operators have the same precedence but do not associate with each other. You must use parentheses to show how they are grouped. Write `p ==> q <== r` as either `(p ==> q) <== r` or `p ==> (q <== r)`.

In contrast, `p ==> q ==> r` is `p ==> (q ==> r)` and `p <== q <== r` is `(p <== q) <== r`.

See this section for more information.

Error: Ambiguous use of && and ||. Use parentheses to disambiguate.

```
const b := true && false || true
```

The `&&` and `||` operators have the same precedence but do not associate with each other. You must use parentheses to show how they are grouped. Write `p && q || r` as either `(p && q) || r` or `p && (q || r)`.

Error: chaining not allowed from the previous operator

```
const c := 1 in {1} == true
```

Chained operations are a sequence of binary operations without parentheses: *a op b op c op d op e*. But there are limitations on which operators can be in one chain together.

In particular, the relational operators `in` and `!in` may not be part of a chain. Use parentheses as necessary to group the operations.

Error: a chain cannot have more than one != operator

```
const c := 1 != 2 != 3
```

Chained operations are a sequence of binary operations without parentheses: *a op b op c op d op e*. But there are limitations on which operators can be in one chain together.

In particular for this error message, one cannot have chains that include more than one `!=` operator.

Error: this operator cannot continue this chain

```
const c := {} !! {} != {}
```

Chained operations are a sequence of binary operations without parentheses: *a op b op c op d op e*. But there are limitations on which operators can be in one chain together.

In particular for this error message, the designated operator is not permitted to extend the existing chain.

Error: this operator chain cannot continue with an ascending operator

```
const c := 2 > 3 < 4
```

Chained operations are a sequence of binary operations without parentheses: *a op b op c op d op e*. But there are limitations on which operators can be in one chain together.

In particular for this error message, one cannot have chains that include both less-than operations (either `<` or `<=`) and greater-than operations (either `>` or `>=`).

Error: this operator chain cannot continue with a descending operator

```
const c := 2 < 3 > 4
```

Chained operations are a sequence of binary operations without parentheses: *a op b op c op d op e*. But there are limitations on which operators can be in one chain together.

In particular for this error message, one cannot have chains that include both less-than operations (either `<` or `<=`) and greater-than operations (either `>` or `>=`).

Error: can only chain disjoint (!!) with itself

```
const c := 2 < 3 !! 4
```

Chained operations are a sequence of binary operations without parentheses: *a op b op c op d op e*. But there are limitations on which operators can be in one chain together.

In particular for this error message, a disjoint operator (`!!`) can appear in a chain only if all the operators in the chain are disjoint operators.

As described here, `a !! b !! c !! d` means that `a`, `b`, `c`, and `d` are all mutually disjoint (which is a different rewriting of the chain than for other operations).

Error: this operator cannot be part of a chain

```
const c := 2 < 3 in 4
```

The operators `in` and `!in` are relational operators, but they may not occur in a chain. Use parentheses if necessary. Such expressions are usually not type-correct in any case.

Error: invalid relational operator

```
const s : set<int>
const r := s ! s
```

The parser is expecting a relational expression, that is, two expressions separated by a relational operator (one of `==`, `!=`, `>`, `>=`, `<`, `<=`, `!!`, `in`, `!in`). But the parser saw just a `!`, which could be the beginning of `!=`, `!!`, or `!in`, but is not continued as such. So perhaps there is extraneous white space or something else entirely is intended.

Error: invalid relational operator (perhaps you intended "!!" with no intervening whitespace?)

```
const s : set<int>
const r := s ! ! s
```

The parser is expecting a relational expression, that is, two expressions separated by a relational operator (one of `==`, `!=`, `>`, `>=`, `<`, `<=`, `!!`, `in`, `!in`). But the parser saw two `!` separated by white space. This is possibly meant to be a `!!` operator, but it could also just be an illegal expression.

Error: Ambiguous use of &, |, ^. Use parentheses to disambiguate.

```
const i: int := 5 | 6 & 7
```

The bit-operators `&`, `|`, and `^` have the same precedence but do not associate with each other. So if they are used within the same expression, parentheses have to be used to show how they are grouped. The example `5 | 6 & 7` should be written as either `(5 | 6) & 7` or `5 | (6 & 7)`.

Error: too many characters in character literal

```
const c := '🚀'
```

A character literal can only contain a single value of the built-in char type. When –unicode-char is disabled, the char type represents UTF-16 code units, so this means a character literal can only contain a character that can be represented with a single such unit, i.e. characters in the Basic Multilingual Plane. The rocket ship emoji ('🚀'), for example, is encoded with two surrogate code points.

This can be fixed by enabling the –unicode-char mode, as that defines char as any Unicode scalar value, but be aware that it may change the meaning of your program.

More detail is given here and here.;

Error: binding not allowed in parenthesized expression

```
method m() {
  var c := ( 4 := 5 );
}
```

Bindings of the form `x := y` are used in map-display expressions, in which case they are enclosed in square brackets, not parentheses. `var c := ( 4 := 5 )` should be `var c := map[ 4 := 5 ]`.

Error: A forming expression must be a multiset

A set/iset/multiset display expression can have two forms. One form is a list of values enclosed by curly braces: `var c := multiset{1, 2, 2, 3}`. The other appears as a conversion operation: `var c := multiset(s)`. However, this second form can only be used to convert a set to a multiset.

In the current parser, however, this error message is unreachable, so if it appears please report the error. The tests that check for this error case are already known to be false by previous testing.

Error: seq type expects only one type argument

```
const c := seq<int,int>(5, i=>i)
```

The built-in `seq` (sequence) type takes one type parameter, which in some situations is inferred. That type parameter is the type of the sequence elements.

Error: a map comprehension with more than one bound variable must have a term expression of the form 'Expr := Expr'

```
const s := map x, y | 0 <= x < y < 10 :: x*y
```

A map comprehension defines a map, which associates a value with each member of a set of keys. The full syntax for a map comprehension looks like `map x | 0 <= x < 5 :: x*2 := x*3` which maps the keys `0, 2, 4, 6, 8` to the values `0, 3, 6, 9, 12` respectively.

One can abbreviate the above syntax to expressions like `map x | 0 <= x < 5 :: x*3`, which is equivalent to `map x | 0 <= x < 5 :: x := x*3`. The missing expression before the `:=` is just the declared identifier.

One can also have multiple variables involved as in `map x, y | 0 < x < y < 5 :: 10*x+y := 10*y+x`, which defines the mappings `(12=>21, 13=>31, 14=>41, 23=>32, 24=>42, 34=>43)`.

But when there are multiple variables, one cannot abbreviate the `:=` syntax with just its right-hand expression, because it is not clear what the left-hand expression should be.

Incorrect text like `const s := map x, y | 0 <= x < y < 10 :: x*y` should be written as `const s := map x, y | 0 <= x < y < 10 :: f(x,y) := x*y` for some `f(x,y)` that gives a unique value for each pair of `x,y` permitted by the range expression (here `0 <= x < y < 10`).

Error: LHS of let-such-that expression must be variables, not general patterns

```
datatype Data = D(i: int, j: int)
const c: int := var Data(d,dd) :| d == 10 && dd == 11; d
```

The let-such-that expression initializes a variable to some value satisfying a given condition. For example, one might write `const cc := var x: int :| x <= 10; x`, where `cc` would get some value `x` satisfying `x < 10`.

For simplicity, however, Dafny requires the variables being initialized to be simple names, not patterns.

Error: a variable in a let expression should be initialized using ':=', ':-', or ':|', not '='

```
method m() {
  var x := var y = 5; y*y;
}
```

Like local variables, let variables are initialized with `:=` (and sometimes with `:-` or `:|`), but not with `=`. In Dafny `=` is used only in type definitions.

Error: ':-' can have at most one left-hand side

```
datatype Outcome<T> =
  | Success(value: T)
  | Failure(error: string)
{ predicate IsFailure() { this.Failure? }
  function PropagateFailure<U>(): Outcome<U>
    requires IsFailure()
  { Outcome<U>.Failure(this.error) // this is Outcome<U>.Failure(...)
  }

  function Extract(): T requires !IsFailure() { this.value }
}

function m(): Outcome<int> { Outcome<int>.Success(0) }
function test(): Outcome<int> {
  var rr, rrr :- m(); Outcome.Success(1)
}
```

Within a function, the `:-` operator is limited to a most one left-hand-side and exactly one-right-hand-side.

Error: ':-' must have exactly one right-hand side

```
datatype Outcome<T> =
  | Success(value: T)
  | Failure(error: string)
{ predicate IsFailure() { this.Failure? }
  function PropagateFailure<U>(): Outcome<U>
    requires IsFailure()
  { Outcome<U>.Failure(this.error) // this is Outcome<U>.Failure(...)
  }

  function Extract(): T requires !IsFailure() { this.value }
}

function m(): Outcome<int> { Outcome<int>.Success(0) }
function test(): Outcome<int> {
```

```
  var rr :- m(), 44; Outcome.Success(1)
}
```

This error only occurs when using the elephant operator `:-` in conjunction with failure-compatible types and in the context of a let-or-fail expression.

In contrast to the let expression (`:=`), which allows multiple parallel initializations, the let-or-fail expression (`:-`) is implemented to only allow at most a single left-hand-side and exactly one right-hand-side.

Error: a set comprehension with more than one bound variable must have a term expression

```
const s := set x, y | 0 <= x < y < 10
```

A set comprehension (1) declares one or more variables, (2) possibly states some limits on those variables, and (3) states an expression over those variables that are the values in the set.

If there is no expression, then the expression is taken to be just the *one* declared variable. For instance one could write `set b: bool`, which is equivalent to `set b: bool :: b` and would be the set of all `bool` values. Another example is `set x: nat | x < 5, which is equivalent to` set x: nat | x < 5:: x `and would be the set` {0, 1, 2, 3, 4}`.

But if more than one variable is declared, then there is no natural implicit expression to fill in after the `::` if it is omitted, so some expression is required. The failing example above, for example, might use the expression `x * y`, as in `set x, y | 0 <= x < y < 10 :: x * y`, or any other expression over `x` and `y`.

Warning: opaque is deprecated as an identifier. It will soon become a reserved word. Use a different name.

```
const opaque: int
```

Because of the value to proof success of using `opaque` declarations and `reveal`ing them in appropriate contexts, the word `opaque` is being converted to a reserved keyword, whereas it used to be a normal identifier. Please rename your use of opaque as an identifier to some other name.

Error: invalid name after a '.'

This error message is not reachable in current Dafny. If it occurs, please report an internal bug (or obsolete documentation).

Error: cannot declare identifier beginning with underscore

```
const _myconst := 5
```

User-declared identifiers may not begin with an underscore; such identifiers are reserved for internal use. In match statements and expressions, an identifier that is a single underscore is used as a wild-card match.

**Warning: deprecated style: a semi-colon is not needed here

```
const c := 5;
```

Semicolons are required after statements and declarations in method bodies,
but are deprecated after declarations within modules and types.

Error: incorrectly formatted number

This error can only result from an internal bug in the Dafny parser. The parser recognizes a legitimate sequence of digits (as an integer literal and then passes that string to a library routine to create a BigInteger or BigDecimal. Given the parser logic, that parsing should never fail.

Error: incorrectly formatted number

This error can only result from an internal bug in the Dafny parser. The parser recognizes a legitimate sequence of hexdigits and then passes that string to a library routine to create a BigInteger. Given the parser logic, that parsing should never fail.

Error: incorrectly formatted number

This error can only result from an internal bug in the Dafny parser. The parser recognizes a legitimate Dafny decimal number and then passes that string to a library routine to create a BigDecimal. Given the parser logic, that parsing should never fail.

Error: invalid *entity*

This “invalid something” message where the something is typically the name of an internal parser non-terminal means that the text being parsed is a badly malformed instance of whatever parser entity was being parsed. This is an automatically generated message by the CoCo parser generator for a situation in which no specific recovery or a more informative error message has been implemented.

The only advice we can give is to carefully scrutinize the location of the error to see what might be wrong with the text. If you think this is a common or confusing enough occurrence to warrant special error handling, please suggest the improvement, with this sample code, to the Dafny team.

Error: Malformed *template* pragma: *#source*

```
const s := @"
#line S
"
```

This pragma syntax is no longer supported. If this message is seen, please report it to the Dafny development team. The Dafny scanner once supported pragmas of the form `#line <lineno> <filename>`, with the filename optional. This message indicates that the pragma was not readable, most likely because the line number was not a parsable numeral.

Error: Unrecognized pragma: *#source*

```
const s := @"
# I love hashes
"
```

This pragma syntax is no longer supported. If this message is seen, please report it to the Dafny development team. The Dafny scanner saw a pragma – the first character of the line is a # character. But it is not one that the scanner recognizes. The only pragma ever recognized was `#line`.

Error: [internal error] Parser exception: *message*

This error indicates an internal crashing bug in Dafny. Please report it with as much of the source code that causes the problem as possible.

Miscellaneous Errors and Warnings

Warning: constructors no longer need 'this' to be listed in modifies clauses

```
class A {
  constructor () modifies this {}
}
```

The purpose of a constructor is to initialize a newly allocated instance of a class. Hence it always modifies the `this` object. Previously it was required to list `this` in the modifies clause of the constructor to specify this property, but now `this` is always implicitly a part of the modifies clause. If the constructor only modifies its own object (as is the very common case) then no explicit modifies clause is needed at all.

Error: \u escape sequences are not permitted when Unicode chars are enabled

```
const c := '\u0000'
```

When using the option `--unicode-chars=true` all unicode characters are written with `\\U`, not `\\`.

Error: \U{X..X} escape sequence must have at most six hex digits

```
const c := '\U{00AABBCC}'
```

When using the option `--unicode-chars=true` all unicode characters are written with `\\U`, not `\\`.

Error: \U{X..X} escape sequence must be less than 0x110000

```
const c := '\U{110011}'
```

Unicode characters (with `--unicode-chars=true`) are defined only up through `0x110000`.

Error: \U{X..X} escape sequence must not be a surrogate

```
const c := '\U{D900}'
```

The allowed range of unicode characters in Dafny excludes the surrogate characters in the range 0xD800 .. 0xE000.

Error: \U escape sequences are not permitted when Unicode chars are disabled

```
const c := '\U{D000}'
```

With `--unicode-chars=false`, all unicode characters are written with a lower-case `u`.

Error: Option *name* unrecognized or unsupported in ':options' attributes.

```
module {:options "ZZZ"} M {}
```

Dafny allows certain attributes to be defined for specific scopes (such as modules) in a program. Only a small set of attributes are permitted to be locally redefined. The error message is indicating that the given attribute is not one of them. Note that some attributes (e.g. `--unicode-char`) must be consistent across the whole program.

Name and Type Resolution Errors and Warnings

Error: ghost variables such as *name* are allowed only in specification contexts. *name* was inferred to be ghost based on its declaration or initialization.

```
method m() {
  ghost var i := 6;
  var j := i;
  print j;
}
```

By their nature, ghost variables and ghost expressions generally may not affect the compiled code. So ghost variables may not be used in any non-ghost (compiled) statement. Note that variables can be ghost because they are explicitly declared to be ghost or because they are initialized with a value that is derived from a ghost expression.

Error: a *what* is allowed only in specification contexts

```
datatype A = A(x: int, ghost y: int)
method m(a: A) returns (r: int) {
    return a.y;
}
```

Ghost expressions, including ghost fields or destructors, are allowed only in ghost code.

Error: a *what* with ghost parameters can be used as a value only in specification contexts

```
class A {
  function f(ghost i: int): int {0}
}
method m(a:A)
{
  print a.f;
}
```

Functions may have some (or all) formal parameters be ghost. Such parameters can only be used in ghost expressions within the function body. There are limits though on where such a function may be used. For example, passing the value of the function itself (not a call of the function) is restricted to ghost contexts.

Error: *what* '*name*' can be used only in specification contexts

```
datatype D = J | ghost K(i: int)
method m(d:D)
  requires d.K?
{
  print d.i;
}
```

A datatype may have ghost constructors, but accessing the value of one of the fields (destructors) of such a ghost constructor is a ghost operation. Consequently an expression like `d.i` (where `i` is a destructor of a ghost constructor for the datatype of which `d` is a value) is allowed only in ghost contexts.

Error: in a compiled context, update of *deconstructors* cannot be applied to a datatype value of a ghost variant (ghost constructor *constructor*)

```
datatype D = A | ghost B(c: int)
method m(d:D)
  requires d.B?
{
  print d.(c := 0);
}
```

Datatypes may have ghost variants (where the datatype constructor is itself declared ghost), but constructing or updating such variants is a ghost operation. Consequently such expressions may not be present in compiled code.

Error: a call to a *kind* is allowed only in specification contexts_hint_

```
twostate function f(): int { 42 }
method m() returns (r: int) {
  r := f();
}
```

`twostate` declarations, extreme predicates, and prefix lemmas are always ghost (even without a `ghost` keyword). Thus they may never be used outside a ghost context.

Error: a call to a ghost *what* is allowed only in specification contexts (consider declaring the *what* without the 'ghost' keyword)

For Dafny 4:

```
ghost function f(): int { 42 }
method m() returns (a: int)
{
  a := f();
}
```

A ghost function can only be called in a ghost context; assigning to an out-parameter is always a non-ghost context. If you declare the function to be compilable, then it can be used in a non-ghost context. In Dafny 3 a non-ghost function is declared as `function method` (and just `function` is ghost); in Dafny 4, `function` is non-ghost and `ghost function` is ghost (like the declarations for methods). See the reference manual on –function-syntax.

Error: a call to a ghost *what* is allowed only in specification contexts (consider declaring the *what* with 'function method')

For Dafny 3:

```
function f(): int { 42 }
method m() returns (a: int)
{
  a := f();
}
```

A ghost function can only be called in a ghost context; assigning to an out-parameter is always a non-ghost context. If you declare the function to be compilable, then it can be used in a non-ghost context. In Dafny 3 a non-ghost function is declared as `function method` (and just `function` is ghost); in Dafny 4, `function` is non-ghost and `ghost function` is ghost (like the declarations for methods). See the reference manual on –function-syntax.

Error: ghost constructor is allowed only in specification contexts

```
datatype D = A | ghost C
method m(i: int) returns (r: D){
  if i == 0 { r := A; }
  if i != 0 { r := C; }
}
```

A datatype can have a mix of non-ghost and ghost constructors, but the ghost constructors may only be used in ghost contexts. For example, a ghost constructor cannot be assigned to a non-ghost out-parameter or used in the then- or else-branch of a non-ghost if statment.

Error: old expressions are allowed only in specification and ghost contexts

```
class A {}
method m(a: A) returns (r: A){
  r := old(a);
}
```

The `old` construct is only used in ghost contexts. Typically using `old` forces an expression to be ghost. But in situations where it is definitely not a ghost context, such as assigning to a non-ghost out-parameter or the actual argument for a non-ghost formal parameter, then `old` cannot be used.

Error: an expression of type '*type*' is not run-time checkable to be a '*type*'

Error: fresh expressions are allowed only in specification and ghost contexts

```
class A {}
method m(a: A) returns (b: bool) {
  b := fresh(a);
}
```

The `fresh` construct is only used in ghost contexts. Typically using `fresh` forces an expression to be ghost. So `fresh` cannot be used in situations where it is definitely not a ghost context, such as assigning to a non-ghost out-parameter or the actual argument for a non-ghost formal parameter.

Error: unchanged expressions are allowed only in specification and ghost contexts

```
class A {}
method m(a: A) returns (b: bool){
  b := unchanged(a);
}
```

The `unchanged` construct is only used in ghost contexts. Typically using `unchanged` forces an expression to be ghost. So `unchanged` cannot be used in situations where it is definitely not a ghost context, such as assigning to a non-ghost out-parameter or the actual argument for a non-ghost formal parameter.

Error: rank comparisons are allowed only in specification and ghost contexts

```
datatype D = A | B
method m(dd: D)
{
  var d := A;
  print d < dd;
}
```

The `<` operator for two datatype values denotes *rank comparison*. That is, the right-hand operand must be structurally deeper than the left for the result of the operator to be true. However, this operation is always a ghost operation and is never compiled. So it cannot appear in a non-ghost context.

Error: prefix equalities are allowed only in specification and ghost contexts

```
codatatype Stream = SNil | SCons(head: int, tail: Stream)
const s: Stream
const t: Stream
const b := s == #[1] t
```

The `==#[k]` syntax is *prefix equality* on two values of the same codatatype. It means that the two values have the same prefix of k values. Such operations are not compilable and only allowed in ghost contexts.

Error: *what* in non-ghost contexts must be compilable, but Dafny's heuristics can't figure out how to produce or compile a bounded set of values for '*name*'

```
const s := iset i: int :: i*2
```

Implicit iterations over unbounded ranges are not compilable. More typically a *range* predicate is given that limits the range of the local variable. The dafny tool analyzes this predicate, using various heuristics, to find lower and upper bounds by which to constrain the range. If the heuristics fail, then dafny does not know how to, and will not, compile the code. Where possible, adding in a range predicate, even if it is a superset of the actual range, can give the compiler enough hints to construct a compiled version of the program.

Error: match expression is not compilable, because it depends on a ghost constructor

```
datatype D = A | ghost B
method m(dd: D)
{
  print match dd { case A => 0 case B => 1 };
}
```

If one of the cases in a match expression uses a ghost constructor, then the whole match expression is ghost. That match expression cannot then be used in a compiled context, such as a print statement.

Error: newtype's base type is not fully determined; add an explicit type for '*name*'

Error: subset type's base type is not fully determined; add an explicit type for '*name*'

Error: shared destructors must have the same type, but '*name*' has type '*type*' in constructor '*name*' and type '*type*' in constructor '*name*'

```
datatype D = A(x: int) | B(x: bool)
```

In defining a datatype, two constructors can both refer to a common destructor, but if they do, the two instances must be declared with the same type. To correct this, either (a) the writer intends there to be two different destructor values, but accidentally used the same

name, in which case change the name of one of them, or (b) they are intended to be the same, in which case a common type must be chosen.

Error: literal (*literal*) is too large for the bitvector type *type*

```
const b: bv4 := 30
```

An integer literal can be converted implicitly to a value of a bitvector type, but only if the integer literal is in the range for the target type. For example, the type `bv4` has 4 bits and holds the values 0 through 15 inclusive. So a `bv4` can be initialized with a value in that range. Negative values are allowed: a value of -n corresponds to the bit vector value which, when added to the bitvector value of n, gives 0. For bv4, -n is the same as 16-n.

Error: unary minus (-*num*, type *type*) not allowed in case pattern

```
const d: bv4
const c := match d { case -0 => 0 case _ => 1 }
```

In a case value of a match expression with a bitvector type, the literals in the cases may not be negative literals, even though those may be used as bitvector literals in some other places in Dafny.

Error: type of type parameter could not be determined; please specify the type explicitly

Error: type of bound variable '*name*' could not be determined; please specify the type explicitly

Error: type of bound variable '*name*' ('*type*') is not allowed to use type ORDINAL

Warning: the quantifier has the form 'exists x :: A ==> B', which most often is a typo for 'exists x :: A && B'; if you think otherwise, rewrite as 'exists x :: (A ==> B)' or 'exists x :: !A || B' to suppress this warning

```
ghost const c := exists i: int :: true ==> true
```

The implication `A ==> B` is true if `A` is false or `B` is true. In a `forall` statement one might write, for example, `0 <= i < 10 ==> a[i] == 0`, claiming that the array element is 0 for the first 10 array elements. But if one wrote `exists i :: 0 <= i < 10 ==> a[i] == 0` then a value of 10 for `i` satisfies the predicate. More often one means `exists i :: 0 <= i < 10 && a[i] == 0`, that is, is there an `i` between 0 and 10 for which the array element is 0. This is such a common mistake that this warning warns about it and asks for a syntax that explicitly states that the writer means it.

Error: type parameter '*name*' (inferred to be '*type*') to the *kind* '*name*' could not be determined

Error: type parameter '*name*' (passed in as '*type*') to the *kind* '*name*' is not allowed to use ORDINAL

Error: type parameter '*name*' (inferred to be '*type*') in the function call to '*name*' could not be determined

Error: type parameter '*name*' (passed in as '*type*') to function call '*name*' is not allowed to use ORDINAL

Error: the type of the bound variable '*var*' could not be determined

Error: a type cast to a reference type (*type*) must be from a compatible type (got *type*); this cast could never succeed

Error: a type test to '*type*' must be from a compatible type (got '*type*')

Error: a non-trivial type test is allowed only for reference types (tried to test if '*type*' is a '*type*')

```
type Small = i: nat | i < 10
const i := 10
const b := i is Small
```

The `is` type test is currently somewhat limited in Dafny, and more limited than the companion `as` conversion. In particular, `is` is not allowed to test that a value is a member of a subset type.

Warning: the type of the other operand is a non-null type, so this comparison with 'null' will always return '*bool*'*hint*

```
class C {}
function f(): C
method m(c: C) {
  var b: bool := f() != null;
  var a: bool := c != null;
}
```

Dafny does have a `null` value and expressions of types that include `null` can have a `null` value. But in Dafny, for each class type `C` there is a corresponding type `C?`; `C` does not include `null`, whereas `C?` does. So if an expression `e` having type `C` is compared

against `null`, as in `e == null`, that comparison will always be `false`. If the logic of the program allows `e` to be sometimes `null`, then it should be declared with a type like `C?`.

Warning: the type of the other operand is a *what* of non-null elements, so the *non_inclusion test of 'null' will always return '_bool'*

```
class C {}
method m(c: seq<C>, cc: seq<C?>) {
  var bb := null in cc;  // OK
  var b  := null in c;   // Warning
}
```

This error refers to the `in` (or `!in`) operation and notes that the test is whether `null` is in the given container. But the elements of the container are of a type that does not include `null`, so the given test will always be `false` (or `true`). Either the type of the container's elements should be a nullable type (a `C?` instead of a `C`) or the test is unnecessary.

Warning: the type of the other operand is a map to a non-null type, so the inclusion test of 'null' will always return '*bool*'

```
trait T {}
const m: map<T,T>
const c := null in m
```

The operation is testing whether `null` is a member of the domain of the map value given as the right-hand operand. But the domain of that map type (the first type parameter) is a non-null type. So this test will trivially always fail (for `in`) or succeed (for `!in`). If it is actually the case that the map's domain is allowed to contain `null` then the domain type should be a nullable type like `T?`. If it is not the case that null could be in the domain, then this test is not needed at all.

Error: the type of this *var* is underspecified

Error: an ORDINAL type is not allowed to be used as a type argument

```
type X<T>
method m(c: X<ORDINAL>) {
}
```

The ORDINAL type corresponds to a mathematical type "larger" than the natural numbers. That is, there are ORDINALs that are larger than any `nat`. Logical reasoning with ORDINALs is tricky and a bit counter-intuitive at times. For logical implementation reasons, Dafny limits where ORDINALs can be used; one restriction is that the ORDINAL type may not be a type argument.

Error: the value returned by an abstemious function must come from invoking a co-constructor

```
codatatype D = A | B
function {:abstemious} f(): int {0}
```

Abstemious functions are not documented. Please report occurences of this error message.

Error: an abstemious function is allowed to invoke a codatatype destructor only on its parameters

```
codatatype EnormousTree<X> = Node(left: EnormousTree, val: X, right: EnormousTr
ghost function {:abstemious} BadDestruct(t: EnormousTree): EnormousTree
{
  Node(t.left, t.val, t.right.right)  // error: cannot destruct t.right
}
```

Abstemious functions are not documented. Please report occurences of this error message.

Error: an abstemious function is allowed to codatatype-match only on its parameters

```
codatatype EnormousTree<X> = Node(left: EnormousTree, val: X, right: EnormousTr
ghost function {:abstemious} BadMatch(t: EnormousTree): EnormousTree
{
  match t.right  // error: cannot destruct t.right
  case Node(a, x, b) =>
    Node(a, x, b)
}
```

Abstemious functions are not documented. Please report occurences of this error message.

Error: an abstemious function is allowed to codatatype-match only on its parameters

Abstemious functions are not documented. Please report occurences of this error message.

Error: an abstemious function is not allowed to check codatatype equality

```
codatatype D = A | B(i: bool)
ghost function {:abstemious} f(d: D, e: D): D { B(d == e) }
```

Abstemious functions have some restrictions. One of these is that an abstemious function may not invoke test of equality over codatatypes, even though this is allowed for non-abstemious ghost functions. See the reference manual for more information on using abstemious functions.

Error: expect statement is not allowed in this context (because this is a ghost method or because the statement is guarded by a specification-only expression)

```
method m(ghost b: bool)
{
  var x := 0;
  if b { expect x == 0; }
}
```

`expect` statements are not allowed in ghost contexts; use an `assert` statement instead. Ghost context can be explicitly clear, such as the body of a method or function declared `ghost`. But a ghost context can also be implicit, and not so obvious: if part of a statement, such as the condition of an if statement or loop or the expression being matched in a match statement, is ghost, then the rest of the statement may be required to be ghost.

Error: print statement is not allowed in this context (because this is a ghost method or because the statement is guarded by a specification-only expression)

```
method m(ghost b: bool)
{
  if b { print true; }
}
```

`print` statements are not allowed in ghost contexts, because they have external effects. Ghost context can be explicitly clear, such as the body of a method or function declared `ghost`. But a ghost context can also be implicit, and not so obvious: if something ghost is part of a statement, such as the condition of an `if` statement or loop or the expression being matched in a match statement, then the rest of the statement may be required to be ghost.

In addition, methods must be marked with the `{:print}` attribute if it has `print` statements or calls methods marked with `{:print}` and `--track-print-effects` is enabled. See the reference manual discussion on :print and tracking print effects.

Error: ghost-context *kind* statement is not allowed to *kind* out of non-ghost *target*

```
method m(i: int) {
  ghost var b: bool := true;
  label x: while i > 0 {
    while (b) {
      break x;
    }
  }
}
```

A `break` or `continue` statement might be in a ghost loop or block, for example, because the condition of the loop is ghost. It then may direct the flow of control to break out of some outer enclosing loop or block or continue an iteration in an enclosing loop. If that enclosing loop or block is non-ghost, we have the situation of ghost code affecting the flow of control of non-ghost code. Consequently a ghost break or continue statement must have as its target some enclosing ghost block or loop.

Error: *kind* statement is not allowed in this context (because it is guarded by a specification-only expression)

```
method m() {
  ghost var b: bool := true;
  if b { return; }
}
```

If the condition of a `if` or `match` statement is a ghost expression, then the whole statement is considered ghost. And then the statement can contain no substatements that are forbidden to be ghost. `return` and `yield` stastements are never ghost, so they cannot appear in a statement whose guarding value is ghost.

Error: cannot assign to *var* in a ghost context

```
method m(ghost c: int)
{
  var x := 7;
  if (c == 1) { x :| x < 8; }
}
```

No changes to non-ghost variables may occur in ghost contexts. Ghost context can be implicit, such as the branches of an if-statement whose condition is a ghost expression.

Error: non-ghost *var* cannot be assigned a value that depends on a ghost

```
method m(ghost c: int)
{
```

```
  var x := 8;
  x :| x < c;
}
```

In a assign-such-that statement, the LHS gets some value that satisfies the boolean expression on the RHS. If the LHS is non-ghost, then the RHS must be non-ghost, because non-ghost code may not depend on ghost code.

Error: assumption variable must be of type 'bool'

```
method m() {
  ghost var {:assumption} b: int;
}
```

Variables marked with `{:assumption}` must have bool type. See the reference manual for more detail on the use of this attribute.

Error: assumption variable must be ghost

```
method m() {
  var {:assumption} b: bool;
}
```

Variables marked with `{:assumption}` must be ghost. See the reference manual for more detail on the use of this attribute.

Error: assumption variables can only be declared in a method

Variables marked with `{:assumption}` must be declared within methods. See the reference manual for more detail on the use of this attribute.

Error: in *proof*, calls are allowed only to lemmas

```
method n() {}
method m()
{
  var i: int;
  assert true by {
    n();
  }
}
```

Proof contexts are blocks of statements that are used to construct a proof. Examples are the bodies of lemma, the by-blocks of assert-by statements and calc statements. A proof context is a ghost context. In ghost context, no methods may be called, even ghost methods. Only lemmas may be called.

Error: only ghost methods can be called from this context

```
method n() {}
ghost method m()
{
  n();
}
```

The body of a ghost method is a ghost context. So if there are any method calls in that body, they must be ghost. Lemmas and ghost functions may also be called.

Error: actual out-parameter *parameter* is required to be a ghost variable

```
method n() returns (r: int, ghost s: int) {}
method m(a: array<int>) returns (r: bool)
  requires a.Length > 1
{
  var x: int;
  var y: int;
  x, y := n();
  a[0] := 0;
}
```

The method being called returns a ghost value as one of its out-parameters. The variable receiving that value is required to be ghost as well. Note that out-parameters are numbered beginning with 0.

This message can also arise when the receiving left-hand-side expression is an array element (e.g. a[0]). This is not permitted at all because arrays are never ghost.

Error: actual out-parameter *parameter* is required to be a ghost field

```
class A { var a: int }
method n() returns (r: int, ghost s: int) {}
method m(a: A) returns (r: bool)
{
  var x: int;
  var y: int;
  x, a.a := n();
}
```

The method being called returns a ghost value as one of its out-parameters. The left-hand-side expression receiving that value is required to be ghost as well. In this case, the LHS expression is a field of an object; the field itself must be ghost, not simply the whole object. Note that out-parameters are numbered beginning with 0.

Error: a loop in *context* is not allowed to use 'modifies' clauses

```
class A { var a: int }
method m(aa: A)
  modifies aa
{
  assert true by {
    var i:= 10;
    while (i > 0)
      decreases i
      modifies aa
    {
    i := i - 1;
    }
  }
}
```

Proof contexts are blocks of statements that are used to construct a proof. Examples are the bodies of lemma, the by-blocks of assert-by statements and calc statements. A proof context is a ghost context. One of the rules for ghost contexts is that nothing on the heap may be modified in ghost context. Consequently there is no need for a modifies clause for loops that might be used to support the proof in the `by` block.

Error: 'decreases *' is not allowed on ghost loops

```
method m()
  decreases *
{
  ghost var c := 10;
  while 0 <= c
    invariant 0 <= c <= 10
    decreases *
  {
    c := c - 1;
  }
}
```

A while loop is ghost if its controlling condition is a ghost expression. Similarly, a for loop is ghost if the range over which the index variable ranges is ghost. Ghost code is meant to aid proofs; for sound proofs any constructs in the ghost code must be terminating. Hence, indications of non-terminating loops, that is, `decreases *`, are not permitted.

This does mean that the specifier has to do the work of designing a valid terminating condition and proving it.

Error: a loop in *proof* is not allowed to use 'modifies' clauses

```
class A {  }
lemma m(j: int, a: A) {
  var i := j;
  while (i > 0)
    modifies a
  {
  }
}
```

–>

A proof context, such as the body of a lemma, is ghost context and thus is not allowed to modify anything on the heap. If there is nothing that may be modified, then there is no need for a `modifies` clause for a loop. Note that the `modifies` clause does not list any local variables that are changed in a loop in any case.

Error: a ghost loop must be terminating; make the end-expression specific or add a 'decreases' clause

Error: *proof* is not allowed to perform an aggregate heap update

```
lemma p(a: array<int>)
{
  forall i | 0 <= i < a.Length { a[i] := 0; }
}
```

Proof contexts, such as bodies of lemmas or by-blocks, cannot perform any changes to the heap. In particular that disallows assignments to array elements using a `forall` aggregate update.

Error: forall statements in non-ghost contexts must be compilable, but Dafny's heuristics can't figure out how to produce or compile a bounded set of values for '*name*'

Error: a modify statement is not allowed in *proof*

```
class A {  }
lemma m(a: A)
 {
  modify a;
}
```

A proof context, such as the body of a lemma or a `by` block, is ghost context and thus is not allowed to modify anything on the heap. If there is nothing that may be modified, then there is no need for a `modify` statement in such a context.

Error: *proof* is not allowed to use 'new'

```
class A {  }
lemma m(a: A)
{
  var aa := new A;
}
```

A proof context, such as the body of a lemma or a `by` block, is ghost context and thus is not allowed to modify anything on the heap. That includes allocating new things in the heap, as a `new` expression does. Typically a proof uses expressions that are value types (sequences, sets, maps) to formulate the proof and not heap operations.

Error: *proof* is not allowed to make heap updates

```
class A { ghost var k: int }
lemma m(a: A)
{
  a.k := 9;
}
```

An update to a field of a heap object is not allowed in a proof context such as the body of a lemma. This is the case even if the field in question is a ghost field of its containing class or trait.

Error: assignment to *kind* is not allowed in this context, because *reason*

```
class A { var a: int }
lemma lem(aa: A) {
  aa.a := 1;
}
method m(aa: A) {
  ghost var b := true;
  if b { aa.a := 1; }
}
```

This message can occur in many program situations: the fault is an assignment to a field in the heap that is not permitted because the assignment statement occurs in a ghost context within a generally non-ghost environment. A common example is the then or else branch of a `if` statement that is deemed a ghost context because the controlling condition for the loop is a ghost expression. Similar situations arise for loops and match statements.

Error: the result of a ghost constructor can only be assigned to a ghost variable

```
class A { constructor I() {} ghost constructor Init(i: int) {} }
method m() returns (a: A)
{
  a := new A.Init(23);
}
```

Classes may have ghost constructors along with regular, non-ghost constructors. However, ghost constructors may only be called in ghost context, including that the newly allocated object be assigned to a ghost location (such as a ghost variable).

Error: tail recursion can be specified only for methods that will be compiled, not for ghost methods

```
ghost method {:tailrecursion} m(n: nat) returns (r: nat)
{
  if n == 0 { r := 0; } else { r := m(n-1); }
}
```

Tail-recursion is a compilation optimization that transforms recursive calls into loops. So it is only relevant to compiled code; ghost functions and methods need not (and may not) specify a `{:tailrecursion}` attribute.

Error: sorry, tail-call optimizations are not supported for mutually recursive methods

```
method g(n: nat) returns (r: nat) { r := f(n); }
method {:tailrecursion} f(n: nat) returns (r: nat)
{
  if n == 0 { r := 0; } else { r := g(n-1); }
}
```

Tail-recursion is a compilation optimization that transforms recursive calls into loops. However, the implementation currently in the `dafny` tool does not support identifying and optimizing tail-recursion opportunities in mutually recursive functions.

Error: this recursive call is not recognized as being tail recursive, because it is followed by non-ghost code

```
method {:tailrecursion} m(n: nat) returns (r: nat)
{
  r := m(n-1);
```

```
  r := 0;
}
```

Tail-recursion is a compilation optimization that transforms recursive calls into loops. As the name somewhat implies, a situation amenable to tail-recursion optimization is one in which the recursive call of the function or method is the last thing that happens in the body of the function or method. If other code follows the recursive call, the tail-recursion transformation cannot be applied.

Error: the recursive call to '*name*' is not tail recursive, because the assignment of the LHS happens after the call

Error: a recursive call inside a loop is not recognized as being a tail call

```
method {:tailrecursion} m(n: nat) returns (r: nat)
{
  while n > 0 { r := m(n-1); }
}
```

Tail-recursion is a compilation optimization that transforms recursive calls into loops. As the name somewhat implies, a situation amenable to tail-recursion optimization is one in which the recursive call of the function or method is the last thing that happens in the body of the function or method. That is certainly not the case for calls within loops, so such calls may not be attributed with `{:tailrecursion}`.

Error: a recursive call inside a forall statement is not a tail call

Error: a recursive call in this context is not recognized as a tail call

Error: the recursive call to '*name*' is not tail recursive because the actual out-parameter is not the formal out-parameter *formal*

```
method {:tailrecursion} m(n: nat) returns (r: nat)
{
  var x := m(n-1);
}
```

Tail-recursion is a compilation optimization that transforms recursive calls into loops. Part of the pattern that is amenable to this optimization is that the last thing happening in the body of a recursive method (or function) is a call of the recursive function with the value returned being assigned to the out-parameter.

Error: the recursive call to '*name*' is not tail recursive because the actual out-parameter *out* is not the formal out-parameter *formal*

```
method {:tailrecursion} m(n: nat) returns (r: nat, s: nat)
{
  var x, y := m(n-1);
}
```

Tail-recursion is a compilation optimization that transforms recursive calls into loops. Part of the pattern that is amenable to this optimization is that the last thing happening in the body of a recursive method (or function) is a call of the recursive function with the value returned being assigned to the out-parameter.

Error: the recursive call to '*name*' is not tail recursive because the actual type argument *typeparam* is not the formal type parameter '*formal*'

```
method {:tailrecursion} m<T,U>(n: nat) returns (r: nat)
{
  if n == 0 { return 0; } else { r := m<U,U>(n-1); }
}
```

If the method that is marked for tail-recursion has type parameters, then the recursive calls of that method must have the same type parameters, in the same order. Note that type parameter positions start from 0.

Error: tail recursion can be specified only for functions that will be compiled, not for ghost functions

```
ghost function {:tailrecursion} f(n: nat): int
{
  if n == 0 then 0 else f(n-1)
}
```

Tail-recursion is a compilation optimization that transforms recursive calls into loops. So it is only relevant to compiled code; ghost functions and methods need not (and may not) specify a `{:tailrecursion}` attribute.

Error: sorry, tail-call optimizations are not supported for mutually recursive functions

```
function g(n: nat): nat { f(n) }
function {:tailrecursion} f(n: nat): nat
{
  if n == 0 then 0 else g(n-1)
}
```

Tail-recursion is a compilation optimization that transforms recursive calls into loops. However, the implementation currently in the `dafny` tool does not support identifying and optimizing tail-recursion opportunities in mutually recursive functions.

Error: if-then-else branches have different accumulator needs for tail recursion

Error: cases have different accumulator needs for tail recursion

Error: to be tail recursive, every use of this function must be part of a tail call or a simple accumulating tail call

```
function {:tailrecursion} f(n: nat): nat
{
  if n == 0 then n else var r := f(n-1); r
}
```

```
function {:tailrecursion} f(n: nat): nat
{
  if n < 2 then n else f(n-2)+f(n-1)
}
```

Tail-recursion is a compilation optimization that transforms recursive calls into loops. The Dafny compiler analyzes the code for recognizable, not too complex patterns. One complexity that defies analysis is too many calls of the same recursive function; another is using the function as a value, rather than in a function call.

Error: a newtype ('*type*') must be based on some non-reference, non-trait, non-arrow, non-ORDINAL, non-datatype type (got *type*)

```
datatype D = A | B
newtype T = D
```

In the current definition of the Dafny language, newtypes may only be based on numeric types: int and real and subtypes and newtypes based on them. This may change in the future.

Error: newtype constraint must be of type bool (instead got *type*)

```
newtype T = i: int | i
```

The expression after the vertical bar must be a boolean condition. The values of the basetype that satisfy this condition are the members of the newtype. This is different than, say, a set comprehension like `set i: int :: i*2` where the expression after the `::` gives the elements of the set directly.

Error: subset type constraint must be of type bool (instead got *type*)

```
type T = i: int | i
```

The expression after the vertical bar must be a boolean condition. The values of the basetype that satisfy this condition are the members of the subset type. This is different than, say, a set comprehension like `set i: int :: i*2` where the expression after the `::` gives the elements of the set directly.

Error: witness expression must have type '*type*' (got '*type*')

```
type T = i: int | 100 < i < 102 witness true
```

In some definitions of subset types or newtypes, Dafny needs an example value to know that the type is not empty. That value is called a `witness` and is a specific value that can be proved to be a member of the type. Since it is a member of the newly defined type, and hence of the basetype, the witness may not be an expression of some different type.

Error: type of unary - must be of a numeric or bitvector type (instead got *type*)

```
datatype D = A | B
const d := A
const x := -d
```

The unary - (negation) operator acts only on `int`, `real`, and bitvector values and values of types based on those types.

Error: type of 'null' is a reference type, but it is used as *type*

```
const i: int := null
```

`null` is a literal value (like `true` and `1`) whose type is a reference type. So it can only be used in contexts that can accept a reference value, such as assignment to a variable of nullable reference type. Primitive types like `boolean`, `int`, `real`, `char`, `string`, and datatypes do not have any value `null` (and there are no types like `string?` or `D?` for a datatype `D`).

Error: integer literal used as if it had type *type*

This error is not yet documented.

Error: real literal used as if it had type *type*

This error is not yet documented.

Error: ‘this’ is not allowed in a ‘static’ context

```
class A {}
method m() {
  var a: A := this;
}
```

As in some other programming languages, `this` in Dafny refers to the object that contains the method in which the identifier `this` is used. However, the containing object is an implicit argument to a method only if it is an *instance* method, not if it is a *static* method; so `this` cannot be used in static methods.

A method in a class is instance by default and static only if it is explicitly declared so. A method declared at the top-level or as a direct member of a module is implicitly static (and cannot be instance).

Error: Identifier does not denote a local variable, parameter, or bound variable: *name*

This error message is not yet documented.

Error: Undeclared datatype: *type*

This error message is not yet documented. Please report any source code that provokes it.

Error: The name *type* ambiguously refers to a type in one of the modules *modules* (try qualifying the type name with the module name)

```
module M { type T }
module N { type T }

import opened M
import opened N
const t: T
```

The stated type name has more than one declaration in scope, likely because both have been imported with `opened`. In that case the name must be qualified to indicate which declaration is intended.

Error: Expected datatype: *type*

This error message is not yet documented. Please report any source code that provokes it.

Error: All elements of display must have some common supertype (got *type*, but needed type or type of previous elements is *type*)

```
const d := [4.0, 6]
```

Error: All elements of display must have some common supertype (got *type*, but needed type or type of previous elements is *type*)

```
const d := map[2 := 3, 4.0 := 6]
```

A map display associates a number of domain values with corresponding range values using the syntax *domain value* := *range value*. All the domain values must have the same type or a common supertype.

Error: All elements of display must have some common supertype (got *type*, but needed type or type of previous elements is *type*)

```
const d := map[2 := 3, 4 := 6.0 ]
```

A map display associates a number of domain values with corresponding range values using the syntax *domain value* := *range value*. All the range values must have the same type or a common supertype.

Error: name of module (*name*) is used as a variable

```
module M {}
const c := M
```

All names in Dafny (except for names of export sets) are in one namespace. Names in different scopes can be distinguished by qualification. Names can also be redeclared in a nested scope with different properties. But if a name is visible, it must be used consistent with its declaration. So a name declared as a module cannot be used as a variable, even though it is usually clear from context where module names are used and where expressions are used.

Error: name of type (*type*) is used as a variable

```
type t = int
method m(i: int) {
  t := i;
}
```

All names in Dafny (except for names of export sets) are in one namespace. Names in different scopes can be distinguished by qualification. Names can also be redeclared in a nested scope with different properties. But if a name is visible, it must be used consistent

with its declaration. So a name declared as a type cannot be used as a variable, even though it is usually clear from context where type names are used and where expressions are used.

Error: a two-state function can be used only in a two-state context

```
module M {
  twostate function f(): int
}
const c := M.f()
```

A `twostate` function is a function of two different heaps. Accordingly it can be used only in situations in which there are two states in play (that is, a *two-state* context). One example is in a postcondition (*ensures* clause) where the two states are the states at the beginning and end of the method execution. Another two-state context is the body of a method, where the states are the pre-state of the method and the current state at the location of the call. However, contexts outside of a method, such as initializations of const declarations, are not two-state contexts.

Error: a field must be selected via an object, not just a class name

This error message is not yet documented. Please report any source code that provokes it.

Error: member *name* in type *type* does not refer to a field or a function

This error message is not yet documented. Please report any source code that provokes it.

Error: array selection requires an array_n_ (got *type*)

```
const a: int
const x: ORDINAL
method m()
{
  var c := a[0,0];
}
```

The `a[1,2,3]` syntax denotes the selection of an array element of a multi-dimensional array. So the expression prior to the '[' must have array type with a number of dimensions that matches the number of index expressions between the left and right brackets.

Error: array selection requires integer- or bitvector-based numeric indices (got *type* for index *i*)

```
const a: array2<int>
const c := a['c',0]
```

Multi-dimensional arrays are selected only by integer or bit-vector values. There is no implicit conversion from characters or reals. A value of ORDINAL type may be used if it can be proved that the value is less than the length of the array dimension. Note that the 'index' in the error message is counted from 0.

Error: update requires a sequence, map, or multiset (got *type*)

```
method m(i: int, s: seq<int>)
  requires |s| > 100
{
  var ss := i[1 := 10];
}
```

The update syntax provides a way to produce a slightly modified sequence, multiset, or map: if `s` is a `seq<int>`, then `s[1 := 10]` is a `seq<int>` with the same values at the same positions as `s`, except that the value at position 1 is now 10. It is important to understand that these are *value* types; the original value of `s` is unchanged; rather a new value is produced as a result of the update expression.

Error: datatype update expression requires a root expression of a datatype (got *type*)

```
method m(a: int)
{
var c := a.(x := 0);
}
```

The `.(` syntax indicates a *datatype update expression*. The expression before the `.(` should be a value of some datatype, but Dafny's type inference found it to be something else.

Error: non-function expression (of type *type*) is called with parameters

```
method m(i: int)
{
  var k := i(9);
}
```

The syntax `f(a,b,c)` is an example of a call of a function or method `f`, with, in this case, three actual arguments, which must correespond to the formal arguments in the definition of `f`. This syntax is only legal in an expression if the expression prior to the left parenthesis is a function, and not something else. It need not be just an identifier; it could be a expression, such as a lambda expression: `((f:int)=>42)(1)`.

Error: wrong number of arguments for function application (function type '*type*' expects *number*, got *number*)

```
function f(): int { 0 }
const k := f(1,2);
```

This message indicates that in some function call the number of actual arguments does not match the number of formal parameters (as given in the function definition). Usually the actuals and formals must match exactly, but Dafny does allow for optional and named arguments with default values. In those cases, the number of actual arguments may be less than the number of formal parameters. (cf. Parameter Bindings in Reference Manual)

Error: type mismatch for argument *i* (function expects *type*, got *type*)

This error message is not yet documented. Please report any source code that provokes it.

Error: sequence construction must use an integer-based expression for the sequence size (got *type*)

```
const s := seq(true, i=>i)
```

The `seq(10, i=>i)` kind of sequence constructor creates a sequence whose size is the value of the first argument and whose elements are filled by applying the given function to the appropriate number of `nat` values (beginning with 0). Accordingly the first argument must be a `nat` and the second a function from `nat` to values of the element type of the sequence.

Error: sequence-construction initializer expression expected to have type '*type*' (instead got '*type*')*hint*

```
const s := seq(10, 20)
```

The `seq(10, i=>i)` kind of sequence constructor creates a sequence whose size is the value of the first argument and whose elements are filled by applying the given function to the appropriate number of `nat` values (beginning with 0). Accordingly the first argument must be a `nat` and the second a function from `nat` to values of the element type of the sequence.

Error: can only form a multiset from a seq or set (got *type*)

```
const m:= multiset(42)
```

The `multiset` function converts a seq or set to a multiset of the same element type. So the argument must be one of those types, and not, say an `int` value.

Error: the argument of a fresh expression must denote an object or a set or sequence of objects (instead got *type*)

```
method m() {
  var i: int;
  assert fresh(i);
}
```

The result of the `fresh` predicate is true if the argument has been allocated since the pre-state of the two-state context containing the call. Thus the argument must be of a type that is allocatable, such as a class type — but not value types like `bool` or `int` or datatypes. The argument may also be a set or sequence of such allocatable objects.

Error: logical/bitwise negation expects a boolean or bitvector argument (instead got *type*)

```
const x := !1
```

The logical negation operator (`!`) applies to `bool` and bitvector values, but not to, for example, `int` values without an explicit conversion. In particular there is no conversion between 0 and false or 1 and true.

Error: size operator expects a collection argument (instead got *type*)

```
method m(x: array<int>) {
  var y: int := |x|;
}
```

The |...| operator is the *size* operator and returns the integer that is the size of its argument. Only finite collections (of type `seq`, `set`, `multiset`, `map`) may be the argument of the size operator – not arrays, `iset`, or `imap`.

Error: a *what* definition is not allowed to depend on the set of allocated references

```
class B {}
const bbb: B
predicate p() { allocated(bbb) }
```

A function is allowed to depend on the heap, as if the heap were an implicit parameter to the function. Any such dependence on mutable fields must be declared in the function's reads clause. Dafny enforces that a function's definition (which includes its body and its requires and reads clauses, but not any of its ensures and decreases clauses) adheres to its reads clause. The purpose of the reads clause is to let you determine when the function's value may

have changed. If you invoke F(x) twice on the same parameter x, then you expect to get the same value. But since the heap is an implicit parameter of the function, will F(x) still give the same value if the heap is changed between the two invocations? The reads clause helps answer this question. Suppose the function's reads clause denotes a set of objects R. Then, as long as the fields of the objects in R are the same for the two invocations of F(x), the two invocations will give the same value. Part of this rule is also that the function is not allowed to depend on the "allocation set", that is, the set of objects that are currently allocated. This is convenient, because a method is always allowed to enlarge the allocation set. As an example, consider a function F(x) with an empty reads clause and a method M() with an empty modifies clause. From this, Dafny allows you to prove the assertion in the following code:

```
var tmp := F(x);
M();
assert tmp == F(x);
```

The non-dependence on the allocation set is checked syntactically by the resolver and the reads clause is enforced by the verifier. Although it would be possible to extend Dafny's logic so that functions could depend on the allocation set, this is at present not implemented.

Error: type conversion to an int-based type is allowed only from numeric and bitvector types, char, and ORDINAL (got *type*)

```
const x: int := true as int
```

Not all pairs of types have implicit or even explicit conversions. But there are conversions to int types from numeric types, including the ORDINAL type; for any source type, the value of the numeric expression must be in the range for the int type (if it is a subset type or a newtype). Even `char` values have an integer representation (and thus a representation as an `int`) corresponding to their unicode value.

Error: type conversion to a real-based type is allowed only from numeric and bitvector types, char, and ORDINAL (got *type*)

```
const x: real := true as real
```

Not all pairs of types have implicit or even explicit conversions. But there are conversions to real types from numeric types, including the ORDINAL type; for any source type, the value of the numeric expression must be in the range for the real type (if it is a subset type or a newtype). Even `char` values have an real representation corresponding to their (integer) unicode value.

Error: type conversion to a bitvector-based type is allowed only from numeric and bitvector types, char, and ORDINAL (got *type*)

```
const x: bv1 := true as bv1
```

Not all pairs of types have implicit or even explicit conversions. But there are explicit conversions to bitvector types from numeric types, including the ORDINAL type; for any source type, the value of the numeric expression must be in the range for the bitvector type. Even `char` values have a bitvector representation corresponding to their (integer) unicode value.

Error: type conversion to a char type is allowed only from numeric and bitvector types, char, and ORDINAL (got *type*)

```
const x: char := true as char
```

Not all pairs of types have implicit or even explicit conversions. But there are explicit conversions to the char type from numeric types, including the ORDINAL type; for any source type, the value of the numeric expression must be in the range for the char type. The numeric values for a given `char` is its numeric unicode representation, which (for `--unicode-char=true`) is in the range 0 to 0x10FFFF, inclusive, though there are some sub-ranges that are not valid unicode characters.

Error: type conversion to an ORDINAL type is allowed only from numeric and bitvector types, char, and ORDINAL (got *type*)

```
const x: ORDINAL := true as ORDINAL
```

Not all pairs of types have implicit or even explicit conversions. But there are explicit conversions to the ORDINAL type from numeric types. Even `char` values have an integer representation and ORDINAL value corresponding to their unicode value.

Error: type cast to reference type '*type*' must be from an expression of a compatible type (got '*type*')

```
method m(i: int) {
  var x := i as object;
}
```

The Dafny `as` is a type cast. But Dafny only allows such casts (or checks with `is`) between types that could possibly be cast from one to the other. In this case, something that is not a reference type is attempting to be cast to a type that is a reference type.

Error: type cast to type '*type*' must be from an expression of a compatible type (got '*type*')

```
datatype D = A | B
const c := 0 as D
```

The `as` operator is the type conversion operator. But it is only allowed between an expression and a type if it is syntactically possible for the value to be converted to the type. Some types, such as datatypes, have no conversions to or from other types. Type conversions from a value of a datatype to some other type are always identity functions and are not allowed to be written.

Error: type test for type '*type*' must be from an expression assignable to it (got '*type*')

```
datatype D = A | B
const c := 0 is D
```

The `is` operator is the type test operator. It asks whether the expression on the left is a value of the type on the right. It might be used to see if a value of a trait type is actually a value of some class derived from that trait, or whether a `int` value is actually a value of some int-based subtype. However, the `is` operator is not allowed to be used in cases where the type of the expression on the left means that the expression could never be a value of the type on the right. For example a class value could never be converted to a datatype value, so an `is` between a reference expression and a datatype type is not allowed.

Error: first argument to *op* must be of type bool (instead got *type*)

```
const b := true
const i := 4
const c := i || b
```

The logical operators `||`, `&&`, `==>`, `<==`, `<==>` take only `bool` arguments. Dafny does not have any implicit conversion to or from `bool` values.

Error: second argument to *op* must be of type bool (instead got *type*)

```
const b := true
const i := 4
const c := b ==> i
```

The logical operators `||`, `&&`, `==>`, `<==`, `<==>` take only `bool` arguments. Dafny does not have any implicit conversion to or from `bool` values.

Error: range of quantified variable must be of type bool (instead got *type*)

```
function f(i: set<int>): set<int> { set k: int <- i |  true || k  }
```

In a quantification using the `<-` syntax, the type of the quantified variable is determined by its explicit declaration or by the type of the elements of the container (the right-hand operand). If the quantified variable is used as a `bool` value when it is not a `bool`, this error message occurs.

Error: arguments must have comparable types (got *type* and *type*)

```
datatype D = D()
const z := 0 == D()
```

All types have `==` and `!=` operations between two elements of the same type. And in cases of subtypes and bit-vector types there may be implicit conversions that allow members of two different but related types to be compared. But dissimilar types cannot be compared.

Error: arguments to *op* must have a common supertype (got *type* and *type*)

```
predicate m(x: int, s: set<int>)  { s !! x }
```

The `!!` operator takes sets as operands. The complaint here is likely that one of the operands is not a set.

Error: arguments must be of a set or multiset type (got *type*)

```
const z := 1 !! 2
```

The `!!` operator denotes disjointness of sets. It is only permitted between sets or multisets of the same element type.

Error: arguments to rank comparison must be datatypes (got *type* and *type*)

```
datatype D = D()
class A {}
method m(a: D, b: A) {
  assert a < b;
  assert a > b;
}
```

The `<` and `>` operators are used for traditional numeric comparison, comparison of prefixes in sequences (just `<`), subset relations among sets, and for rank (structural depth) comparisons between values of the same datatype. When used for rank comparison, both operands must be values of the same datatype.

Error: arguments to *expr* must have a common supertype (got *type* and *type*)

```
const x: ORDINAL
const y: int
const z := y < x
const w := y >= x
```

For binary operators, the two operands must be able to be implicitly converted to the same supertype. For example, two different int-based subtypes would be converted to int, or two values of different classes that extend the same trait could be converted to values of that trait. Where Dafny cannot determine such a common supertype, the comparison is illegal and this error message results.

Error: arguments to *op* must be of a numeric type, bitvector type, ORDINAL, char, a sequence type, or a set-like type (instead got *type*)

```
const x: map<int,int>
const z := x < x
```

The `<`, `<=`, `>=`, and `>` operators are used for traditional numeric comparison, comparison of prefixes in sequences (just `<`), and subset relations among sets. But they are not used for comparing maps or reference values.

Error: type of *op* must be of a bitvector type (instead got *type*)

```
const z :=  0 << 1
```

The `<<` and `>>` operators are left- and right-shift operations. They shift a bit-vector value by a given integer number of bits. The left-hand operand must be a value of a bit-vector type. Even int literals are not implicitly converted to bitvectors (because Dafny would not know which bit-vector type to use). An explicit conversion is required.

Error: type of left argument to *op* (*type*) must agree with the result type (*type*)

Error: type of right argument to *op* (*type*) must be an integer-numeric or bitvector type

```
const z :=  (1 as bv4) << true
```

The `<<` and `>>` operators are left- and right-shift operations. They shift a bit-vector value by a given integer number of bits. The right-hand operand must be an integer value, but its type may be an int-based type (such as a subtype) or a bit-vector type.

Error: type of + must be of a numeric type, a bitvector type, ORDINAL, char, a sequence type, or a set-like or map-like type (instead got *type*)

```
datatype D = D()
const z := D() + D()
```

The `+` operand in Dafny is used for traditional numeric addition, for concatenation of sequences, and for unions. But not for all types. There is no `+` for datatypes or references, for example.

Error: type of left argument to + (*type*) must agree with the result type (*type*)

```
const z := 0 + {1}
```

Though the `+` operand applies to many of Dafny's types, the left- and right- operand need to be the same type or convertible to the same type. For example, there is no conversion from a type to a collection of that type.

Error: type of right argument to + (*type*) must agree with the result type (*type*)

```
const z := {1} + 0
```

Though the `+` operand applies to many of Dafny's types, the left- and right- operand need to be the same type or convertible to the same type. For example, there is no conversion from a type to a collection of that type.

Error: type of - must be of a numeric type, a bitvector type, ORDINAL, char, or a set-like or map-like type (instead got *type*)

```
datatype D = D()
const z := D() - D()
```

The `-` operand in Dafny is used for traditional numeric subtraction, for set difference, and key removal from maps. But not for all types. There is no `-` for datatypes or references, for example.

Error: type of left argument to - (*type*) must agree with the result type (*type*)

```
const z := 0 - {1}
```

Though the `-` operand applies the many of Dafny's types, the left- and right- operand need to be the same type or convertible to the same type. For example, there is no conversion from a type to a collection of that type.

Error: map subtraction expects right-hand operand to have type *type* (instead got *type*)

```
function f(mx: map<int,int>, my: map<int,int>): map<int,int> { mx - my }
```

The map subtraction operator takes a map and a set as operands; the set denotes those elements of the map's *domain* that are removed.

Error: type of right argument to - (*type*) must agree with the result type (*type*)

```
const z := {1} - 0
```

Though the `-` operand applies to many of Dafny's types, the left- and right- operand need to be the same type or convertible to the same type. For example, there is no conversion from a type to a collection of that type.

Error: type of * must be of a numeric type, bitvector type, or a set-like type (instead got *type*)

```
function ff(j: map<int,int>): map<int,int> { j * j }
```

The `*` operator is defined to either multiply numeric vales or take the interesection of sets and multisets.

Error: type of left argument to * (*type*) must agree with the result type (*type*)

Error: type of right argument to * (*type*) must agree with the result type (*type*)

```
function ff(i: int, j: real): real { j * i }
```

The types of the two arguments of `*` must be the same (or implicitly convertible to be the same). Typically the result of the expression is determined by the left operand. This message then is stating that the right operand has a different type.

Error: second argument to *op* must be a set, a multiset, a sequence with elements of type int, or a map with domain int (instead got *type*)

```
function ff(i: int, j: real): bool { i in j }
```

The operators `in` and `!in` test membership of a value in a container, so the right-hand operand must be a container of some sort. It may also be a map, in which case membership in the map's domain is checked, but this use is deprecated in favor of `i in m.Keys`,

Error: domain of quantified variable must be a set, multiset, or sequence with elements of type *type*, or a map with domain *type* (instead got *type*)

```
function f(i: int): set<bool> { set k <- i |  k }
```

The syntax `k <- i` means that `k` is a quantified variable whose domain is all the elements of the container `i`. So the type of `i` must be a container, such as a set, in which case the type of `k` is the type of elements of the container. If the right-hand operand is a `map`, then `k` has the type of the domain of the map.

Warning: timeLimitMultiplier annotation overrides timeLimit annotation

```
method {:timeLimitMultiplier 10} {:timeLimit 5} m() {}
```

The {:timeLimitMultiplier n} attribute tells the prover to use a time limit that is the given number times the time limit otherwise specified by the options. The {:timeLimit n} attribute simply sets a new value for the time limit. It does not make sense to have both attributes on the same declaration. One or the other should be removed. If they are both present, just the {:timeLimitMultiplier} is used.

Warning: Argument to 'old' does not dereference the mutable heap, so this use of 'old' has no effect

```
method m(i: int)
{
   var j: int;
   j := 1;
   label L:
   j := 2;
   ghost var k := old@L(j);
}
```

The `old` function, used in specifications and ghost code, indicates that the argument (an expression) is to be evaluated in some previous heap state. So if the argument does not depend on the heap, the `old` has no effect and is likely to lead to misunderstanding. Consequently, Dafny warns against doing so. For example, it has no effect to apply `old` to a local variable; instead one should capture the value of the local variable in a previous state using an otherwise unused ghost variable.

Warning: You have written an assert statement with a negated condition, but the lack of whitespace between 'assert' and '!' suggests you may be used to Rust and have accidentally negated the asserted condition. If you did not intend the negation, remove the '!' and the parentheses; if you do want the negation, please add a space between the 'assert' and '!'.

```
method m() {
  assert!(1 == 1);
}
```

This message warns about a common syntax error for people familiar with Rust. In Rust `assert! e` indicates checking that expression `e` holds. But in Dafny the `!` negates the expression. That is `assert! e` is `assert !e` in Dafny. The syntactic solution to avoid the warning is to either put space between the `assert` and `!` (if the negation is intended) or to omit the `!` (if the negation is not intended).

Warning: _item_s given as :induction arguments must be given in the same order as in the quantifier; ignoring attribute

```
predicate g(i: int, j: int)
function f(): int
  ensures forall i: int, j: int {:induction j,i} | true :: g(i,j)
{ 0 }
```

The `{:induction}` attribute gives some guidance to the prover as to how to construct the induction argument that proves the lemma. The heuristics that infer an induction proof require that the arguments of the attribute be in the same order as the bound variables of the quantifier. If not, the :induction attribute is ignored.

Warning: lemma parameters given as :induction arguments must be given in the same order as in the lemma; ignoring attribute

The `{:induction}` attribute gives some guidance to the prover as to how to construct the induction argument that proves the lemma. The heuristics that infer an induction proof require that the arguments of the attribute be in the same order as the parameters of the lemma. If not, the :induction attribute is ignored.

Warning: invalid :induction attribute argument; expected *what*; ignoring attribute

```
const c := 0
lemma {:induction 42} m(i: int, j: int) ensures i + j == j + i {}
```

The arguments of the `:induction` attribute can be any of the lemma parameters (for a lemma) or bound quantifiers (for a quantifier expression), the `this` keyword, or true or false.

Warning: Constructor name '*pattern*' should be followed by parentheses

```
datatype D = A | B

method m(d: D) {
  match d {
    case A => return;
    case B() => return;
  }
}
```

The pattern following a `case` keyword can be a constructor possibly without arguments or it can be new variable to be bound to the match expression. A parameter-less constructor may be written without parentheses. A simple identifier that happens to be a constructor of right type will be matched in preference as the constructor.

This can lead to surprises if there is a matching constructor that the writer or the reader of the software is unaware of. So it is useful to always write constructors with parentheses, to visually distinguish them from simple identifiers.

The `--warn-missing-constructor-parentheses` option will warn if a constructor is used without parentheses. The solution is to either add parentheses (if a constructor is intended) or rename the identifier (if a fresh bound identifier is intended).

Warning: unusual indentation in *what* (which starts at *location*); do you perhaps need parentheses?

```
function f(b: bool): bool
{
  b &&
  b ==> b // this parses as (b && b) ==> b, but reads as b && (b ==> b)
}
```

Long expressions are best split up across multiple lines; readers often use indentation as a guide to the nesting of subexpressions. For example, parallel conjuncts or disjuncts may be written with the same indentation. This warning occurs when the observed indentation is likely to cause a misunderstanding of the code, particularly of the precedence of operations. It is suggested to use parentheses or to redo the indentation to make the structure of the expression clearer.

Warning: unusual indentation in *what* (which ends at *location*); do you perhaps need parentheses?

```
function f(b: bool): bool
{
  b ==> b &&
  b          // this parses as b ==> (b && b), but reads as (b ==> b) && b
}
```

Long expressions are best split up across multiple lines; readers often use indentation as a guide to the nesting of subexpressions. For example, parallel conjuncts or disjuncts may be written with the same indentation. This warning occurs when the observed indentation is likely to cause a misunderstanding of the code, particularly of the precedence of operations. It is suggested to use parentheses or to redo the indentation to make the structure of the expression clearer.

Error: Methods with the :test attribute may not have input arguments

```
method {:test} m(i: int) {}
```

The test tool runs all the methods that are marked with `{:test}`. The test tool expects two kinds of behavior: (a) the method may abort with a failing `expect` or (b) the method may return a datatype value that has an `IsFailure()` function which returns false. But the test method has no means of selecting inputs to give to a method under test. An experimental test-generation tool is in development, but for the moment, a routine under test may not have input arguments. A typical solution is to write an auxiliary method, marked with `{:test}`, that takes no inputs, and that then calls the desired method with various combinations of inputs.

Error: Methods with the :test attribute can have at most one return value

```
method {:test} m() returns (j: int, k: int)
{
  j, k := 0,0;
}
```

This error only occurs when using `dafny test`. That command executes all methods attributed with `{:test}`, but such test methods are limited to methods with at most one output parameter. Typically that out-parameter has a failure-compatible type whose value is used to indicate whether the test succeeeded or failed. If the type is not a failure-compatible type, the test is presumed to be successful. This is purely an implementation limitation. (cf. Issue 3387)

Warning: The *kind* clause at this location cannot be compiled to be tested at runtime because it references ghost state.

```
method {:extern} m(i: int, ghost j: int)
  requires j == 1
```

The `--test-assumptions` option inserts tests of various contracts and implicit assumptions in a Dafny method, such as the requires and ensures clauses. However, because these inserted tests are compiled into the target program as runtime checks, they cannot refer to any ghost variables. (This mechanism implements Dafny's runtime-assertion checking.)

Warning: Internal: no wrapper for *name*

This message indicates a bug within the `dafny` tool. Please report the program that triggered the message to the Github issues site.

Warning: No :test code calls *name*

This warning indicates that some method marked with {:extern} is not called by any method marked with {:test}. The intention is to check coverage of the programmed unit tests. The warning only appears when using `/testContracts:TestedExterns` with the legacy CLI. It is likely to be removed in a future version of `dafny`.

Error: :print attribute is not allowed on functions

```
function {:print} f(): int { 0 }
```

The `{:print}` attribute is used to mark methods that have a side-effect of printing something, that is, they modify the external environment. Functions, whether ghost or compiled, are intended to have no side-effects at all. Thus Dafny forbids (ghost or compiled) functions from declaring that they have a print side-effect (whether or not print effects are being tracked with `--track-print-effects`).

Error: :print attribute is not allowed on ghost methods

```
ghost method {:print} f() { }
```

The `{:print}` attribute is used to mark methods that have a side-effect of printing something, that is, they modify the external environment. A program's ghost code is not allowed to modify the actual execution of a program. Thus ghost code should not contain print statements. So Dafny forbids ghost methods from declaring that they have a print side-effect (whether or not print effects are being tracked with `--track-print-effects`).

Error: not allowed to override a non-printing method with a possibly printing method (*name*)

```
trait T {
  method m()
```

```
}

class C extends T {
  method {:print} m() {}
}
```

The `{:print}` attribute is used to mark methods that have a side-effect of printing something, that is, they modify the external environment. If a trait declares a method without a `{:print}` attribute, then any implementation of that method may not do any printing, as it must obey the trait's specification. Thus a class implementing that method by an override cannot declare the method as possibly printing something, by giving it a `{:print}` attribute — even if it does not actually do any printing.

Error: :print attribute does not take any arguments

```
method {:print 42} f() { }
```

The `{:print}` attribute is used to mark methods that have a side-effect of printing something. No arguments to the attribute are needed or allowed.

Error: a function-by-method is not allowed to use print statements

```
function f(): int {42 }
by method {
  print "I'm here\n";
  return 42;
}
```

The `{:print}` attribute is used to mark methods that have a side-effect of printing something, that is, they modify the external environment. Functions, whether ghost or compiled, are intended to have no side-effects at all. A function-by-method has a method body that is intended to be a compiled way of computing what the function body expresses. But as a function may not have side-effects, the method body should not have any print statements.

This tracking of print statements (and the forbidding of them by this error message) only happens when the `--track-print-effects` option is enabled; the option is disabled by default.

Error: to use a print statement, the enclosing *method* must be marked with {:print}

```
method m() { print 42; }
```

The `{:print}` attribute is used to mark methods that have a side-effect of printing something, that is, they modify the external environment. So a method that prints something (even transitively) should be marked with `{:print}`. This error indicates that such an attribute is missing – either add the attribute or remove the print statement.

This tracking of print statements (and thus the possibility of this error message) only happens when the `--track-print-effects` option is enabled; the option is disabled by default.

Error: a function-by-method is not allowed to call a method with print effects

```
function m(): int { 42 }
by method { p(); return 42; }

method {:print} p() {}
```

The `{:print}` attribute is used to mark methods that have a side-effect of printing something, that is, they modify the external environment. Functions, whether ghost or compiled, are intended to have no side-effects at all. A function-by-method has a method body that is intended to be a compiled way of computing what the function body expresses. But as a function may not have side-effects, the method body should not have any print statements.

In this case, the method body might print something because a method that is called in the body is attributed with `{:print}` (whether or not it actually does print something).

This tracking of print statements (and thus the possibility of this error message) only happens when the `--track-print-effects` option is enabled; the option is disabled by default.

Error: to call a method with print effects, the enclosing *method* must be marked with {:print}

```
method m() { p(); }

method {:print} p() {}
```

The `{:print}` attribute is used to mark methods that have a side-effect of printing something, that is, they modify the external environment. In this case, the method body might print something because a method that is called in the body is attributed with `{:print}` (whether or not it actually does print something). So the calling method must be marked with `{:print}` as well.

This tracking of print statements (and thus the possibility of this error message) only happens when the `--track-print-effects` option is enabled; the option is disabled by default.

Refinement Errors

Error: *submod* in *module* cannot be imported with "opened" because it does not match the corresponding import in the refinement base *base*.

Error: *submod* in *module* must be imported with "opened" to match the corresponding import in its refinement base *base*.

Error: a type synonym (*name*) is not allowed to replace a *kind* from the refined module (*refined*), even if it denotes the same type

```
module P {
  type T = int
}
module Q refines P {
  type T = int
}
```

A refining declaration must make the base declaration more specific. It may not just be the same declaration (although sometimes that might be convenient).

Error: to redeclare and refine declaration '*name*' from module '*refined*', you must use the refining (`...`) notation

Error: declaration '*name*' indicates refining (notation `...`), but does not refine anything

```
module P {
}
module Q refines P {
  export A ...
}
```

A refining declaration that uses `...` must actually be refining a corresponding declaration in the base module.

Error: can't change if a module export is default (*name*)

```
module P {
  const c: int
  export reveals c
}
module Q refines P {
```

```
  export P ...
}
```

If a base module P has a default export (implicitly named P), then a refining module, Q, may not declare an export set P with the intention of refining it.

Error: a module (*name*) must refine another module

Error: a module export (*name*) must refine another export

Error: a module (*name*) can only refine a module facade

Error: a module (*name*) must refine another module

```
module P {
  type W
}
module Q refines P {
  module W {}
}
```

A submodule M within a module that is refining some base module must refine some submodule M in the base module.

**Error: to be a refinement of *kind* '*name*' declared with (==), *kind* '*name*' must support equality

```
module P {
  type T(==)
}
module Q refines P {
  type T = AlwaysAndForeverMore
  codatatype AlwaysAndForeverMore = Cons(int, AlwaysAndForeverMore)
}
```

The abstract type `T` in module `P` says it supports equality, but its attempted refinement in module `Q` does not. Codatatypes do not generally support equality and so cannot be refinements of equality-supporting abstract types.

**Error: to be a refinement of *kind* '*name*' declared with (!new), *kind* '*name*' must contain no references

```
module P {
  type T(!new)
}
module Q refines P {
```

```
  type T = (int, bool, array<real>)
}
```

The abstract type `T` in module `P` says it contains no reference, but its attempted refinement in module `Q` does. A refining type must support the type characteristics declared of the refined type.

**Error: to be a refinement of *kind* '*name*' declared with (00), *kind* '*name*' must be nonempty

```
module P {
  type T(00)
}
module Q refines P {
  type T = A
  class A { }
}
```

The abstract type `T` in module `P` uses the type characteristic `(00)` to say that it is nonempty. However, a class type is not generally known to be nonempty, so `A` cannot be used as a refinement for `T` in `Q`.

For this particular situation, a possible remedy would be to instead use `type T = A?`, since the nullable type `A?` is known to be nonempty.

**Error: to be a refinement of *kind* '*name*' declared with (0), *kind* '*name*' must support auto-initialization

```
module P {
  type T(0)
}
module Q refines P {
  type T = A
  class A { }
}
```

The abstract type `T` in module `P` uses the type characteristic `(00)` to say that it can be auto-initialized. However, a class type does not support auto-initialization, so `A` cannot be used as a refinement for `T` in `Q`.

For this particular situation, a possible remedy would be to instead use `type T = A?`, since the nullable type `A?` does support auto-initialization.

Error: a *kind* (*name*) cannot declare members, so it cannot refine an abstract type with members

```
module P {
  type T {
    method m() {}
  }
}
module Q refines P {
  type T = int
}
```

When refining a declaration, the refined declaration has all the characteristics of the base declaration, including any members of the base declaration. Some declarations do not have members, such as subset types and type synonyms, so they cannot refine a declaration that has members declared in the base.

Error: an abstract type declaration (*name*) in a refining module cannot replace a more specific type declaration in the refinement base

```
module P {
  type T = int
}
module Q refines P {
  type T ...
}
```

The purpose of refinement is to replace abstract or incompletely defined declarations with more specific declarations. Hence a type that is defined in a refinement base cannot be an abstract type in the refining module.

Error: a *kind* declaration (*name*) in a refinement module can only refine a *kind* declaration or replace an abstract type declaration

```
module P {
  type T = int
}
module Q refines P {
  datatype T ... {}
}
```

The purpose of refinement is to replace abstract or incompletely defined declarations with more specific declarations. The refining declaration needs to be the same kind of declaration as in the base.

Error: an iterator declaration (*name*) in a refining module cannot replace a different kind of declaration in the refinement base

```
module P {
  class I {}
}
module Q refines P {
  iterator I...
}
```

Iterators may only refine iterator declarations.

Error: a type (*name*) in a refining module may not replace an already defined type (even with the same value)

Error: a module (*name*) can only be refined by an alias module or a module facade

Error: a refining iterator is not allowed to add preconditions

```
module P {
  iterator I() yields (x: int)
}
module Q refines P {
  iterator I... requires true
}
```

There are restrictions on what may change when refining an iterator. In particular, no new preconditions may be added even if they are implied by the base declarations's preconditions.

Error: a refining iterator is not allowed to add yield preconditions

```
module P {
  iterator I() yields (x: int)
}
module Q refines P {
  iterator I... yield requires true
}
```

There are restrictions on what may change when refining an iterator. In particular, no new yield preconditions may be added even if they are implied by the base declarations's preconditions.

Error: a refining iterator is not allowed to extend the reads clause

```
module P {
  iterator I() yields (x: int)
```

```
}
module Q refines P {
  iterator I... reads {}
}
```

There are restrictions on what may change when refining an iterator. In particular, no new reads clauses may be added even if they are contained within the base declarations's reads clauses.

Error: a refining iterator is not allowed to extend the modifies clause

```
module P {
  iterator I() yields (x: int)
}
module Q refines P {
  iterator I... modifies {}
}
```

There are restrictions on what may change when refining an iterator. In particular, no new modifies clauses may be added even if they list objects that are contained within the base declarations's modifies clauses.

Error: a refining iterator is not allowed to extend the decreases clause

```
module P {
  iterator I() yields (x: int)
}
module Q refines P {
  iterator I... decreases 1 {}
}
```

There are restrictions on what may change when refining an iterator. In particular, no new decreases clause may be added even if it is the same as or implied by the base declaration's decreases clause.

Error: a const declaration (*name*) in a refining class (*class*) must replace a const in the refinement base

```
module P {
  class A { var c: int }
}
module Q refines P {
  class A ... { const c: int }
}
```

Following the general rule that declarations in the base module are replaced by more specific declarations of the same kind in the refining module, a `const` declaration in the refining module must replace a `const` declaration in the base module (with the same type).

Error: the type of a const declaration (*name*) in a refining class (*class*) must be syntactically the same as for the const being refined

```
module P {
  type T = bool
  class A { const c: bool }
}
module Q refines P {
  class A ... { const c: T }
}
```

The declarations in a refining module must have the same type as in the base module. In fact, to enable easier checking that the type has not changed, the type must be expressed in the same syntactic form in the two declarations. For example, it is not permitted to use a type in a base declaration and an equivalent type synonym for the corresponding variable in the refinement.

Error: a const re-declaration (*name*) can give an initializing expression only if the const in the refinement base does not

```
module P {
  const c := 7
}
module Q refines P {
  const c := 8
}
```

A refined declaration of a `const` may add an initializer, but it cannot replace an initializer declared in the base, even if it is syntactically the same value, such as an explicit literal in one place and an equivalent expression in the other.

Error: a const in a refining module cannot be changed from static to non-static or vice versa: *name*

```
module P {
 class A {const c: int }
}
module Q refines P {
  class A ... { static const c := 7 }
}
```

A `const` declaration that is in a class that is being refined cannot change its static-ness.

Error: a const re-declaration (*name*) is not allowed to remove 'ghost' from the const declaration

```
module P {
  ghost const c := 7
}
module Q refines P {
  const c: int
}
```

A `const` refining declaration cannot change the declaration from `ghost` to non-ghost.

Error: a const re-declaration (*name*) must be to add 'ghost' to the const declaration_info_

```
module P {
  const c := 7
}
module Q refines P {
  const c: int
}
```

A `const` refinement must change something. It can add a `ghost` modifier or it can add an initializer.

Error: a field declaration (*name*) in a refining class (*class*) must replace a field in the refinement base

```
module P {
  type T = int
  class A { const c: int }
}
module Q refines P {
  class A ... { var c: T }
}
```

Following the general rule that declarations in the base module are replaced by more specific declarations of the same kind in the refining module, a `var` declaration in a refining class must replace a `var` declaration in the class of the base module (with the same type).

Error: a field declaration (*name*) in a refining class (*class*) must repeat the syntactically same type as the field has in the refinement base

```
module P {
  type T = int
  class A { var c: int }
}
module Q refines P {
  class A ... { var c: T }
}
```

The field declarations in a refining class must have the same type as in the class in the base module. In fact, to enable easier checking that the type has not changed, the type must be expressed in the same syntactic form in the two declarations. For example, it is not permitted to use a type in a base declaration and an equivalent type synonym in the corresponding place in the refinement.

Error: a field re-declaration (*name*) must be to add 'ghost' to the field declaration

```
module P {
  class A { var c: int }
}
module Q refines P {
  class A ... { var c: int }
}
```

When a class is being refined, any field declaration in the base is copied into the refinement. If there is a redeclaration of the field, it must be to add a `ghost` modifier.

Error: a *kind* declaration (*name*) can only refine a *kind*

```
module P {
  function f(i: int): bool
}
module Q refines P {
  predicate f(i: int) { true }
}
```

The refining declaration must be the same kind of declaration as the base declaration. For example both must be predicates or both must be functions (even if the function is one that returns a `bool`).

Error: a refining *kind* is not allowed to add preconditions

```
module P {
  predicate m(i: nat)
}
```

```
module Q refines P {
  predicate m(i: nat) requires true { true }
}
```

A function in a refining module must be able to be used in the same way as the abstract function in the base module. If there are additional preconditions, then the call contexts for the refined function may be more restricted than for the base function. Thus no new preconditions may be added. This is a syntactic check, so no preconditions can be added even if they are implied by the existing preconditions.

Error: a refining *kind* is not allowed to extend the reads clause

```
module P {
  predicate m() reads {}
}
module Q refines P {
  predicate m() reads this {true }
}
```

A function in a refining module must be able to be used in the same way as the abstract function in the base module. Extending the reads clause with additional objects cxhanges this equivalence and is not allowed. This change is syntactic. The refining function is not allowed to write any reads clauses. It just inherits those from the base declaration. This is the case even if the new reads clause is a repetition or subset of the base declaration.

Error: decreases clause on refining *kind* not supported

```
module P {
  predicate m(i: nat) reads {}
}
module Q refines P {
  predicate m(i: nat) decreases i {true }
}
```

For simplicity, a refining function is not allowed to add decreases clauses to its declaration.

Error: a function in a refining module cannot be changed from static to non-static or vice versa: *name*

```
module P {
  class A { predicate m(i: nat) }
}
module Q refines P {
  class A ... { static predicate m(i: nat) {true } }
}
```

The static-ness of a function declaration in a refining class must be the same as in the base class.

Error: a compiled function cannot be changed into a ghost function in a refining module: *name*

```
module P {
  predicate m(i: nat)
}
module Q refines P {
  ghost predicate m(i: nat)
}
```

If a function is declared as non-ghost in the base module, it may not be declared `ghost` in the refining module.

Error: a ghost function can be changed into a compiled function in a refining module only if the function has not yet been given a body: *name*

```
module P {
  ghost predicate m(i: nat) { true }
}
module Q refines P {
  predicate m(i: nat) { true }
}
```

If a function is declared ghost in a base module, it can then be given a body and declared non-ghost in the refined version of the module. But in the case where the the base declaration already has a body and is `ghost`, the refined declaration cannot then change the function to non-ghost.

Error: the name of function return value '*function*'(*result*) differs from the name of corresponding function return value in the module it refines (*otherresult*)

```
module P {
  function f(a: int): (r: int)
}
module Q refines P {
  function f(a: int): (s: int) { 0 }
}
```

When refining a function, the input and output signature must stay precisely the same – formals, types, and names – including the name of the function result.

Error: the result type of function '*function*' (*type*) differs from the result type of the corresponding function in the module it refines (*othertype*)

```
module P {
  function f(a: int): int { 0 }
}
module Q refines P {
  function f(a: int): bool
}
```

When refining a function, the input and output signature must stay precisely the same – formals, types, and names – including the type of the function result. The types must be syntactically identical; it is not allowed to use a type and an equivalent type synonym, for example.

Error: a refining *kind* is not allowed to extend/change the body

```
module P {
  function f(a: int): int { 0 }
}
module Q refines P {
  function f(a: int): int { 0 }
}
```

When refining a function, the refining declaration can not include a body if the base declaration has a body, even if the texts of the bodies are identical.

Error: a method declaration (*name*) can only refine a method

```
module P {
  function m(a: int): int
}
module Q refines P {
  method m(a: int)  {}
}
```

The refining declaration must be the same kind of declaration as the base declaration. For example both must be methods.

Error: a refining method is not allowed to add preconditions

```
module P {
  method m() {}
}
module Q refines P {
```

```
  method m() requires true {}
}
```

A method in a refining module must be able to be used in the same way as the abstract method in the base module. If there are additional preconditions, then the calling contexts permitted for the refined method may be more restricted than those for the abstract base method. Thus no new preconditions may be added. This is a syntactic check, so no preconditions can be added even if they are implied by the existing preconditions.

Error: a refining method is not allowed to extend the modifies clause

```
module P {
  method m(i: nat)
}
module Q refines P {
  method m(i: nat) modifies {} { }
}
```

A method in a refining module must be able to be used in the same way as the abstract method in the base module. If there are additional objects in the modifies clause, then the usage of the refined module may have more effect than known by the base method signature. Thus no new modifies clauses may be added. This is a syntactic check, so no modifies clauses can be added even if they do not actually add any new objects to the modifies set.

Error: decreases clause on refining method not supported, unless the refined method was specified with 'decreases *'

```
module P {
  method m(a: int)
}
module Q refines P {
  method m(a: int) decreases a {}
}
```

A decreases clause is not permitted in a refining method declaration, even if it is syntactically identical to the clause in the base declaration. The one exception is that if the base declares `decreases *` then the refinement may give a decreases clause (even `decreases` *`). Note that if the refining declaration does not state a decreases clause (the usual case), the refining declaration gets a copy of the base declarations clause.

Error: a method in a refining module cannot be changed from static to non-static or vice versa: *name*

```
module P {
  class A { method m(i: nat) }
}
module Q refines P {
  class A ... { static method m(i: nat) { } }
}
```

There are restrictions on what can be changed in a refinement. In particular, a basic characteristic like being or not being `static` may not change for any kind of declaration.

Error: a ghost method cannot be changed into a non-ghost method in a refining module: *name*

```
module P {
  ghost method m(i: nat)
}
module Q refines P {
  method m(i: nat) { }
}
```

There are restrictions on what can be changed in a refinement. In particular, a basic characteristic like being or not being `ghost` may not change for methods.

Error: a method cannot be changed into a ghost method in a refining module: *name*

```
module P {
  method m(i: nat)
}
module Q refines P {
  ghost method m(i: nat) { }
}
```

There are restrictions on what can be changed in a refinement. In particular, a basic characteristic like being or not being `ghost` may not change for methods.

Error: *what* '*name*' is declared with a different number of type parameters (*count* instead of *oldcount*) than the corresponding *what* in the module it refines

```
module P {
  method m<T>(i: nat)
}
module Q refines P {
```

```
  method m<T,U>(i: nat) { }
}
```

There are restrictions on what can be changed in a refinement. In particular, a basic characteristic like the number of type parameters may not change for any declaration.

Error: type parameters are not allowed to be renamed from the names given in the *kind* in the module being refined (expected '*oldname*', found '*name*')

```
module P {
  method m<T,U>(i: nat)
}
module Q refines P {
  method m<T,W>(i: nat) { }
}
```

There are restrictions on what can be changed in a refinement. In particular, for convenience and readability, the names of type parameters may not change for any declaration.

Error: type parameter '*name*' is not allowed to change the requirement of supporting equality

```
module P {
  method m<T,U(==)>(i: nat)
}
module Q refines P {
  method m<T,U>(i: nat) { }
}
```

There are restrictions on what can be changed in a refinement. In particular, any characteristics of type parameters must remain the same.

Error: type parameter '*name*' is not allowed to change the requirement of supporting auto-initialization

```
module P {
  method m<T,U(0)>(i: nat)
}
module Q refines P {
  method m<T,U>(i: nat) { }
}
```

There are restrictions on what can be changed in a refinement. In particular, any characteristics of type parameters must remain the same.

Error: type parameter '*name*' is not allowed to change the requirement of being nonempty

```
module P {
  method m<T,U(00)>(i: nat)
}
module Q refines P {
  method m<T,U>(i: nat) { }
}
```

There are restrictions on what can be changed in a refinement. In particular, any characteristics of type parameters must remain the same.

Error: type parameter '*name*' is not allowed to change the no-reference-type requirement

```
module P {
  method m<T,U(!new)>(i: nat)
}
module Q refines P {
  method m<T,U>(i: nat) { }
}
```

There are restrictions on what can be changed in a refinement. In particular, any characteristics of type parameters must remain the same.

Error: type parameter '*name*' is not allowed to change variance (here, from '*oldvariance*' to '*variance*')

```
module P {
  type T<+U>
}
module Q refines P {
  type T<U> = int
}
```

There are restrictions on what can be changed in a refinement. In particular, the variance of type parameters must remain the same.

Error: type parameter '*name*' of *what* '*declarationname*' is declared with a different number of type bounds than in the corresponding *what* in the module it refines (expected *oldnum*, found *num*)

```
module A {
  type AbstrType<X extends object>
}

module B refines A {
  type AbstrType<X>
}
```

Error: type bound for type parameter '*name*' of *what* '*declarationname*' is different from the corresponding type bound of the corresponding type parameter of the corresponding *what* in the module it refines (expected '*oldbound*', found '*bound*'

```
module A {
  type AbstrType<X extends object>
}

module B refines A {
  trait Trait { }
  type AbstrType<X extends Trait>
}
```

Error: *kind* '*name*' is declared with a different number of *what* (*num* instead of *oldnum*) than the corresponding *kind* in the module it refines

```
module P {
  method m(i: int)
}
module Q refines P {
  method m(i: int, j: int) {}
}
```

There are restrictions on what can be changed in a refinement. In particular, the number, type and names of formal parameters must remain the same.

Error: there is a difference in name of *kind num* ('*name*' versus '*oldname*') of *kind name* compared to corresponding *kind* in the module it refines

```
module P {
  method m(i: int)
}
module Q refines P {
```

```
  method m(j: int) {}
}
```

There are restrictions on what can be changed in a refinement. In particular, for convenience and readability, the names of formal parameters may not change for any declaration.

Error: *kind* '*name*' of *kind container* cannot be changed, compared to the corresponding *kind* in the module it refines, from non-ghost to ghost

```
module P {
  method m(i: int)
}
module Q refines P {
  method m(ghost i: int) {}
}
```

There are restrictions on what can be changed in a refinement. In particular, ghost-ness of formal parameters may not change for any declaration.

Error: *kind* '*name*' of *kind container* cannot be changed, compared to the corresponding *kind* in the module it refines, from ghost to non-ghost

```
module P {
  method m(ghost i: int)
}
module Q refines P {
  method m(i: int) {}
}
```

There are restrictions on what can be changed in a refinement. In particular, ghost-ness of formal parameters may not change for any declaration.

Error: *kind* '*name*' of *kind container* cannot be changed, compared to the corresponding *kind* in the module it refines, from new to non-new

Error: *kind* '*name*' of *kind container* cannot be changed, compared to the corresponding *kind* in the module it refines, from non-new to new

Error: *kind* '*name*' of *kind container* cannot be changed, compared to the corresponding *kind* in the module it refines, from non-older to older

```
module P {
  class A {}
  predicate m(a: A)
}
```

```
module Q refines P {
  predicate m(older a: A) { true }
}
```

When refining a predicate, a formal parameter may not change from older to non-older or vice versa.

Error: *kind* '*name*' of *kind container* cannot be changed, compared to the corresponding *kind* in the module it refines, from older to non-older

```
module P {
  class A {}
  predicate m(older a: A)
}
module Q refines P {
  predicate m(a: A) { true }
}
```

When refining a predicate, a formal parameter may not change from older to non-older or vice versa.

Error: the type of *kind* '*n*' is different from the type of the same *kind* in the corresponding *thing* in the module it refines ('*name*' instead of '*oldname*')

```
module P {
  method m(a: bool)
}
module Q refines P {
  method m(a: int) {}
}
```

The types in a signature in a refining declaration must be the same as the corresponding types in the base declaration. The types must be syntactically identical. For example one cannot be a type synonym of the other.

Error: a refining formal parameter ('*name*') in a refinement module is not allowed to give a default-value expression

```
module P {
  method m(i: int)
}
module Q refines P {
  method m(i: int := 9) {}
}
```

There are restrictions on what changes can be made in refining a base declaration. When refining methods, one restriction is that the refining declaration may not have default value declarations. The refining method has precisely the same default values as the base declaration.

Error: skeleton statement does not match old statement

Refining statements are no longer supported in Dafny.

Error: assert template does not match inherited statement

Refining statements are no longer supported in Dafny.

Error: expect template does not match inherited statement

Refining statements are no longer supported in Dafny.

Error: assume template does not match inherited statement

Refining statements are no longer supported in Dafny.

Error: if-statement template does not match inherited statement

Refining statements are no longer supported in Dafny.

Error: while-statement template does not match inherited statement

Refining statements are no longer supported in Dafny.

Error: a skeleton while statement with a guard can only replace a while statement with a non-deterministic guard

Refining statements are no longer supported in Dafny.

Error: modify template does not match inherited statement

Refining statements are no longer supported in Dafny.

Error: modify template must have a body if the inherited modify statement does

Refining statements are no longer supported in Dafny.

Error: a refining loop can provide a decreases clause only if the loop being refined was declared with 'decreases *'

Refining statements are no longer supported in Dafny.

Error: while template must have a body if the inherited while statement does

Refining statements are no longer supported in Dafny.

Error: skeleton statement may not be used here; it does not have a matching statement in what is being replaced

Refining statements are no longer supported in Dafny.

Error: yield statements are not allowed in skeletons

Refining statements are no longer supported in Dafny.

Error: *kind* statement in skeleton is not allowed to break outside the skeleton fragment

Refining statements are no longer supported in Dafny.

Error: cannot have assignment statement

Refining statements are no longer supported in Dafny.

Error: cannot have call statement

Refining statements are no longer supported in Dafny.

Error: refinement method cannot assign to variable defined in parent module (‘*name*’)

Refining statements are no longer supported in Dafny.

Error: refinement method cannot assign to a field defined in parent module (‘{0}’)

Refining statements are no longer supported in Dafny.

Error: new assignments in a refinement method can only assign to state that the module defines (which never includes array elements)

Refining statements are no longer supported in Dafny.

Error: assignment RHS in refinement method is not allowed to affect previously defined state

Refining statements are no longer supported in Dafny.

Verification Errors

This section is a work in progress